User Stories

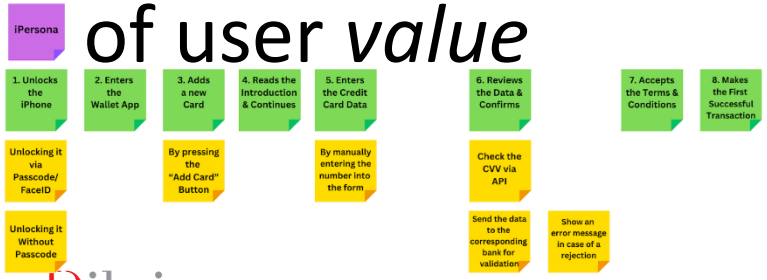
We plan our work in small, customer-centric pieces.

PART 1: THE FOUNDATIONS

Where We Came From: The Limits of *The Spec*



- Massive, detailed specification documents
- Freeze knowledge at the *beginning* of the development
- Often written *in isolation* from the development team
- Rigid, difficult to change, and quickly outdated
- Focus on *features* and *functions*, often losing sight of user *value*



Dibris

A New Approach:
A Placeholder for Conversation

- A user story is a short, simple description of a feature told from the perspective of the person who desires it
  - it describes a *need*
  - it is **not** a detailed specification
- Its primary purpose is to articulate value and initiate a conversation

A user story is an invitation to a conversation
Alistair Cockburn

The Anatomy of a Story

- As a <role>
- I want <goal>
- So that <benefit>

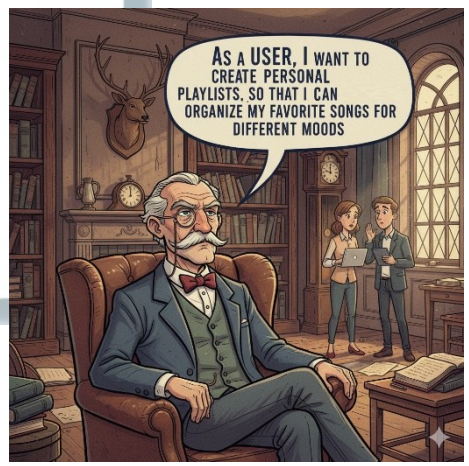
Who are we building this for? (The User)

What are they trying to achieve? (The Action)

Why do they want it? (The Value)

User Story

As a music lover, I want to create personal playslists, so that I can I can organize my favorite songs for different moods.

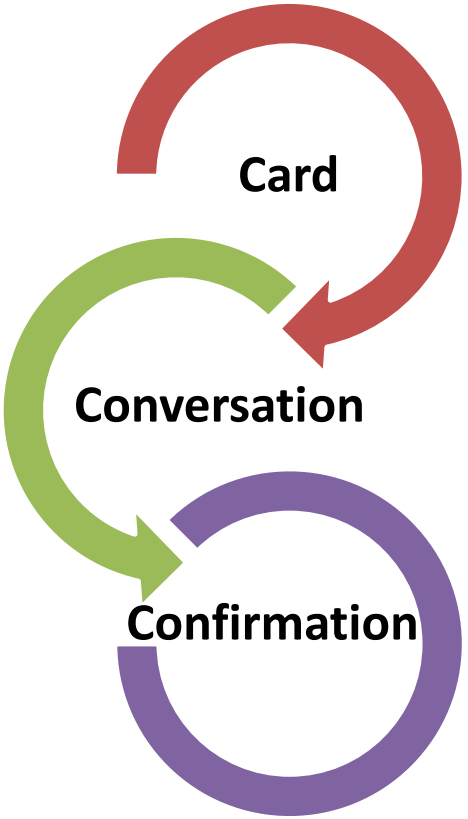




PART 2: THE CRAFT

The Lifecycle of a Story: Card, Conversation, Confirmation

- **Card**: A physical (or digital) token
  - the essence of the requirement
- **Conversation**: A *verbal* exchange between the team, product owner, and stakeholders to flesh out the details
  - building shared understanding
- **Confirmation**: An agreement on how to verify the story is "done" correctly
  - **Acceptance Criteria**



Usually completed over time through many sessions

How We Know We're Done

- Acceptance Criteria (AC) = a set of testable conditions that must be met for the story to be considered complete
  - They transform ambiguity into a clear, shared checklist
  - Written on the back of the card
- **Acceptance Criteria**

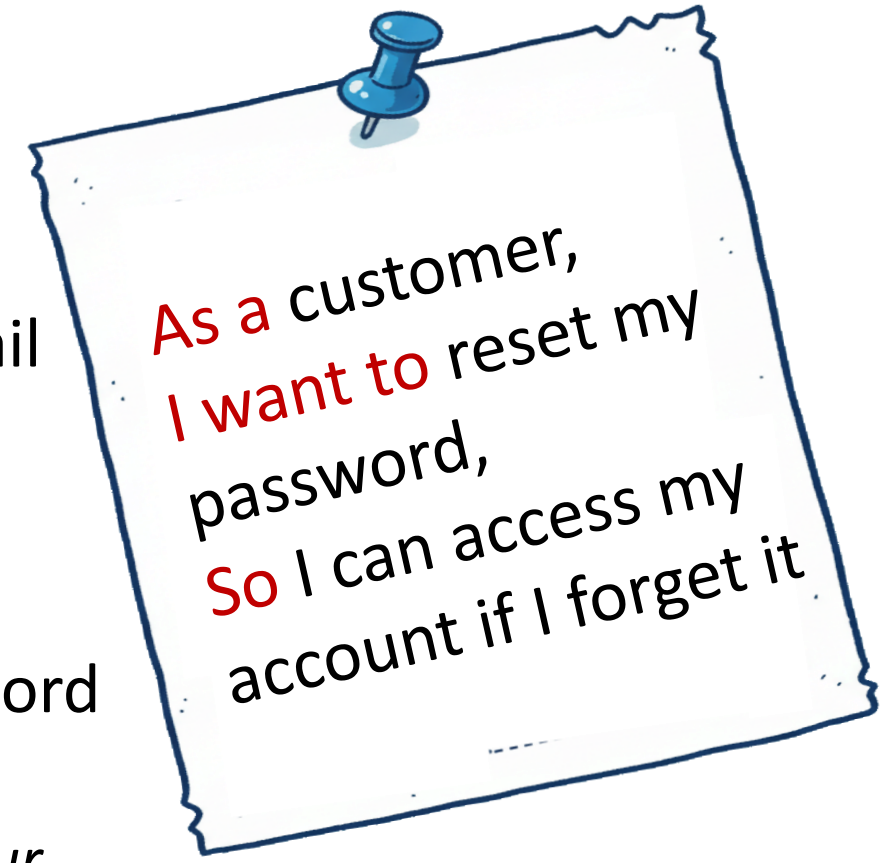
Given I am on the login page,

When I click "Forgot Password,"
Then I am prompted for my email

When I submit a valid email,
Then a confirmation message is shown

Then a unique, single-use password reset link is sent to my email

The link must expire in 1 hour



A Checklist for a Good Story

- **I – Independent** Can be developed and delivered on its own
- **N - Negotiable** Not a rigid contract; details can be discussed
- **V - Valuable** Delivers tangible value to the end-user
- **E - Estimable** The team can give a rough size estimate
- **S - Small** Can be completed within a single development iteration
- **T - Testable** You can define clear acceptance criteria for it

From Big Ideas to Manageable Work

- **Epic**: A very large story that represents a major feature. It's too big to complete in one sprint
- **Story**: A smaller piece of functionality broken out from an Epic that fits the INVEST criteria
- **Task**: A specific technical action needed to complete a story



THE MANAGEMENT

Sizing the Work: The Problem with Time

- **Why not estimate in hours?**
  - Humans are poor at absolute estimation
  - An hour for a senior developer is different from an hour for a junior
  - Time estimates can be perceived as commitments, creating pressure
- **Enter Story Points**
  - A relative measure of effort, complexity, and uncertainty
  - Often uses a modified Fibonacci sequence: 1, 2, 3, 5, 8, 13...
    - sometimes visualized graphically as T-Shirt sizes, country sizes, buckets…
  - A “2” is roughly twice the effort of a “1”, an “8” is a big, complex story
  - A reference story is needed to establish the effort of a “1”

Collaborative Estimation

A consensus-based technique to estimate stories

- **Discuss**
  - Explanations by the Product Owner
  - Q&A with the team
- **Estimate**
  - Each person *privately* chooses a card with a story point value
- **Reveal**
  - All cards are shown at the same time



- **Align**
  - If estimates differ, the highest and lowest are explained
  - To share knowledge
- **Repeat**
  - until a consensus is reached

Deciding What to Build Next

- A simple but powerful framework for prioritizing the product backlog.
- Plot stories on a 2x2 matrix.

Value		
High	Quick Wins	Big Bets
Low	Fill-ins	Money Pits Reconsider
	Low	High **Effort**

PART 4: THE EVOLUTION

The Story's Final Form: Executable Specifications

- What if our Acceptance Criteria could be more than just a checklist?
- What if they could be directly transformed into automated tests?
- This is the core idea behind **Behavior-Driven Development (BDD)**

A Common Language for Business and Tech

- BDD uses a structured, human-readable language called **Gherkin**

  ⇒ understandable by product owners/users

- It follows a Given / When / Then format

  Given: The initial context (the state of the system)

  When: The action the user performs

  Then: The expected outcome or result

  – same structure as a test

  ⇒ test-as-specification for developers

  ⇒ easy to [automatically] translate into a test

BDD Example

Card Title: Add item to cart from search

Card Front

As a shopper,

I want to add an item to my cart from the search results page,

so that I can continue shopping without visiting the product page

Card Back

Given a user is on the search results page for "agile books“

And the "User Stories Applied" book is visible

When the user clicks the "Add to Cart" button for that book

Then the cart icon in the header should display "1 item"

CONCLUSION

The Power of a Story

- **User Stories' Strong Points**
  - **Focus on Value**: They force us to ask "why?"
  - **Foster Collaboration**: They are invitations to a conversation
  - **Enable Iteration**: They are small, manageable units of work
  - **Drive Quality**: They connect directly to testable acceptance criteria

The Flipped Classroom

- On Aulaweb you will find
  - These slides
  - The list of resources everybody **MUST** read.
  - The calendar for next lesson
  - A choice to select your topic
- Each lesson will address one area
  - The volunteers for (topics in) that area will research it further, present it in a 10-minute talk, and lead the discussion
  - After the discussion we will do some hands-on exercise

Focus points

- The Foundations
  - Definition of user story
  - Their role in agile development
  - User roles/personas
  - Good/bad examples



- The craft
  - Lifecycle (e.g., C^3)
  - Acceptance criteria
  - Size



- The Management (why, when, why, how)
  - Estimation
    - Story points
  - Prioritization

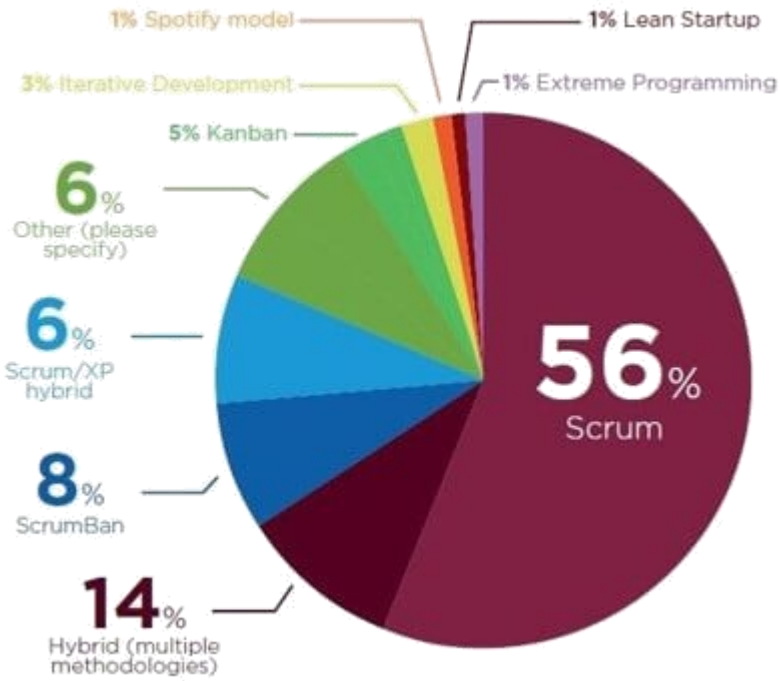


- The Evolution
  - Gherkin
  - Test automation
  - BDD



Chose your Area, the volunteers will coordinate to distribute contents

Scrum



Capstone Project 2025/26

Maura Cerioli

We're losing the relay race

"The… 'relay race' approach to product development…may conflict with the goals of maximum speed and flexibility. Instead a holistic or 'rugby' approach—where a team tries to go the distance as a unit, passing the ball back and forth—may better serve today's competitive requirements."

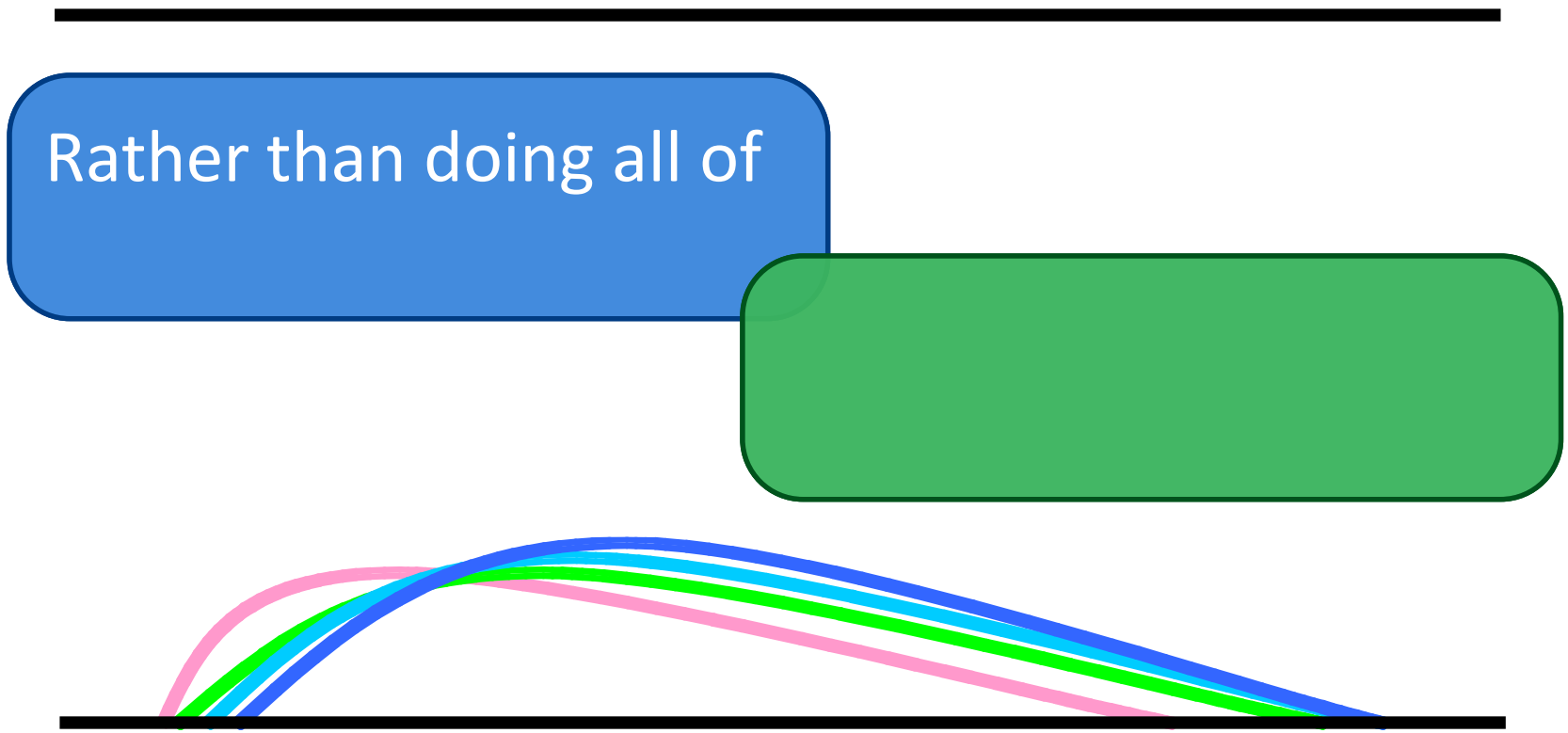
Hirotaka Takeuchi and Ikujiro Nonaka, "The New New Product Development Game", *Harvard Business Review*, January 1986.





Dibris

Sequential vs. overlapping development



Source: "The New New Product Development Game" by Takeuchi and Nonaka. *Harvard Business Review,* January 1986.

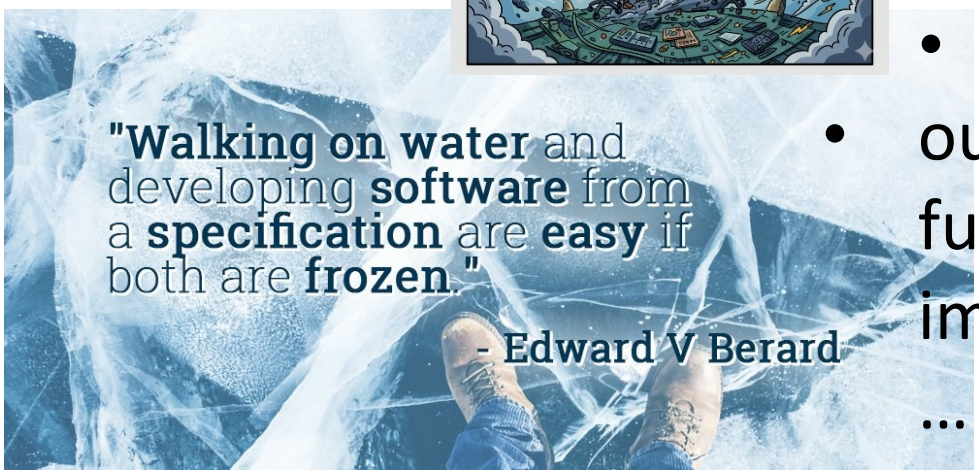


Sprints = iterations

Dichotomy: embracing changes vs. need for stable requirement to get better productivity



- input: **frozen** requirements
  - **only** the sprint drivers
  - including DoD
- time boxed
  - in time: limited by requirements' instability (2 weeks – 1 month)
  - in effort: constant to get a better rhythm
  - risk containement
- output: fully tested correct implementation … or failure

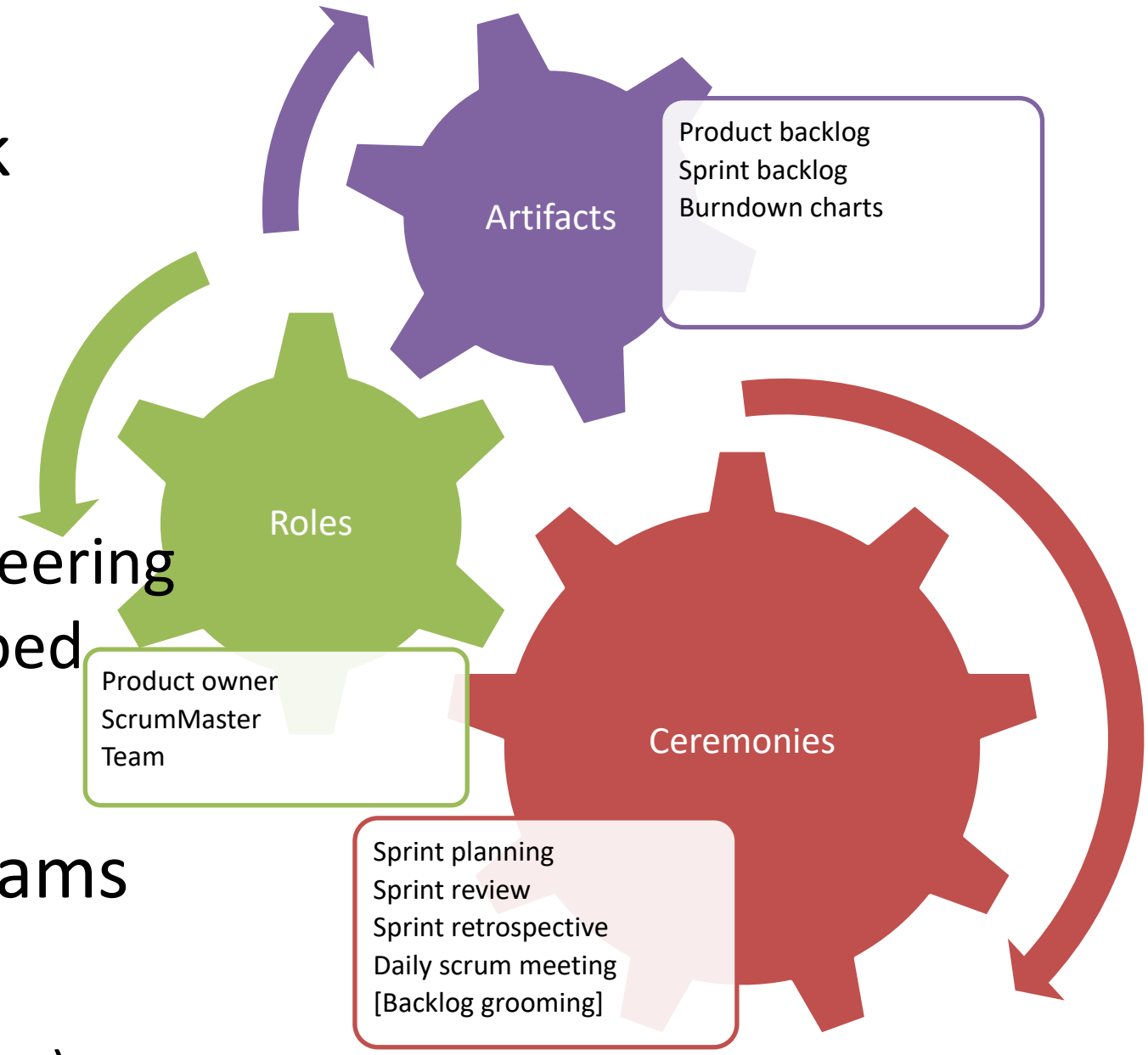




Dibris

Other Main Characteristics

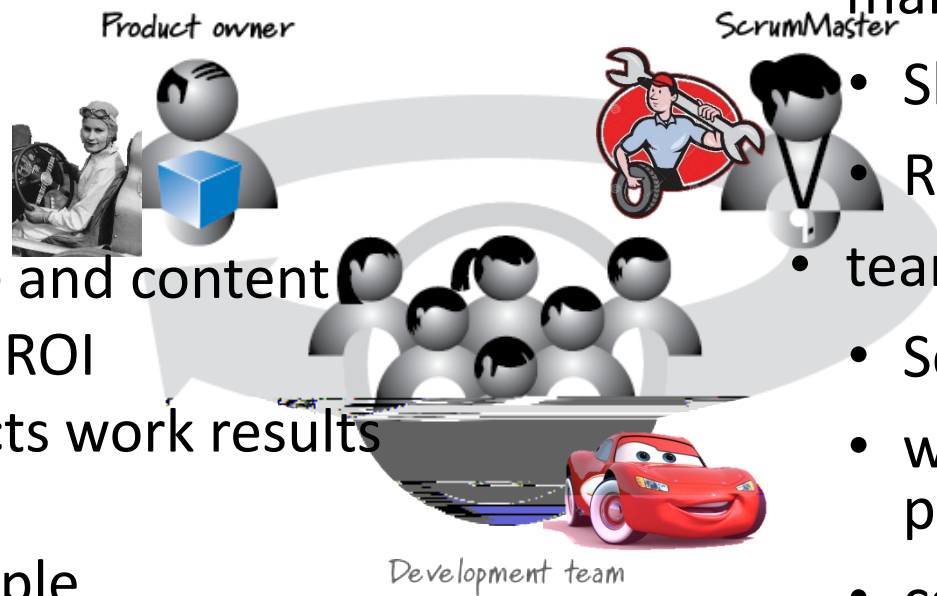
- Scrum framework
  - 3 Roles
  - 4/5 Ceremonies
  - 3 Artifacts
  - No specific engineering practices prescribed
- Self-organizing teams
  - Generative rules
    - (**not** inclusive rules)



Scrum **Key** Roles

Proxy for business stakeholders

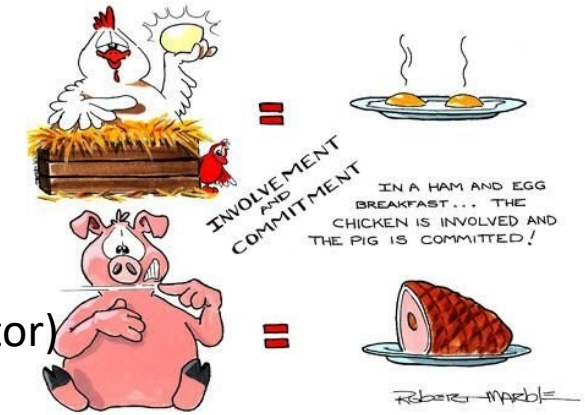
- Decides
  - features
  - priorities
  - release date and content
- Responsible for ROI
- Accepts or rejects work results



Interface team ⇔ management

- management side
  - Shield from interferences
  - Removes impediments
- team side
  - Scrum coach
  - watches for/quenches problems
  - cooperation facilitator

- 5-9 people
- Cross-functional:
  - Programmers, testers, user experience designers, etc.
- Self-organizing
- Membership
  - changes **only** between sprints
  - full-time (but for experts, e.g., database administrator)

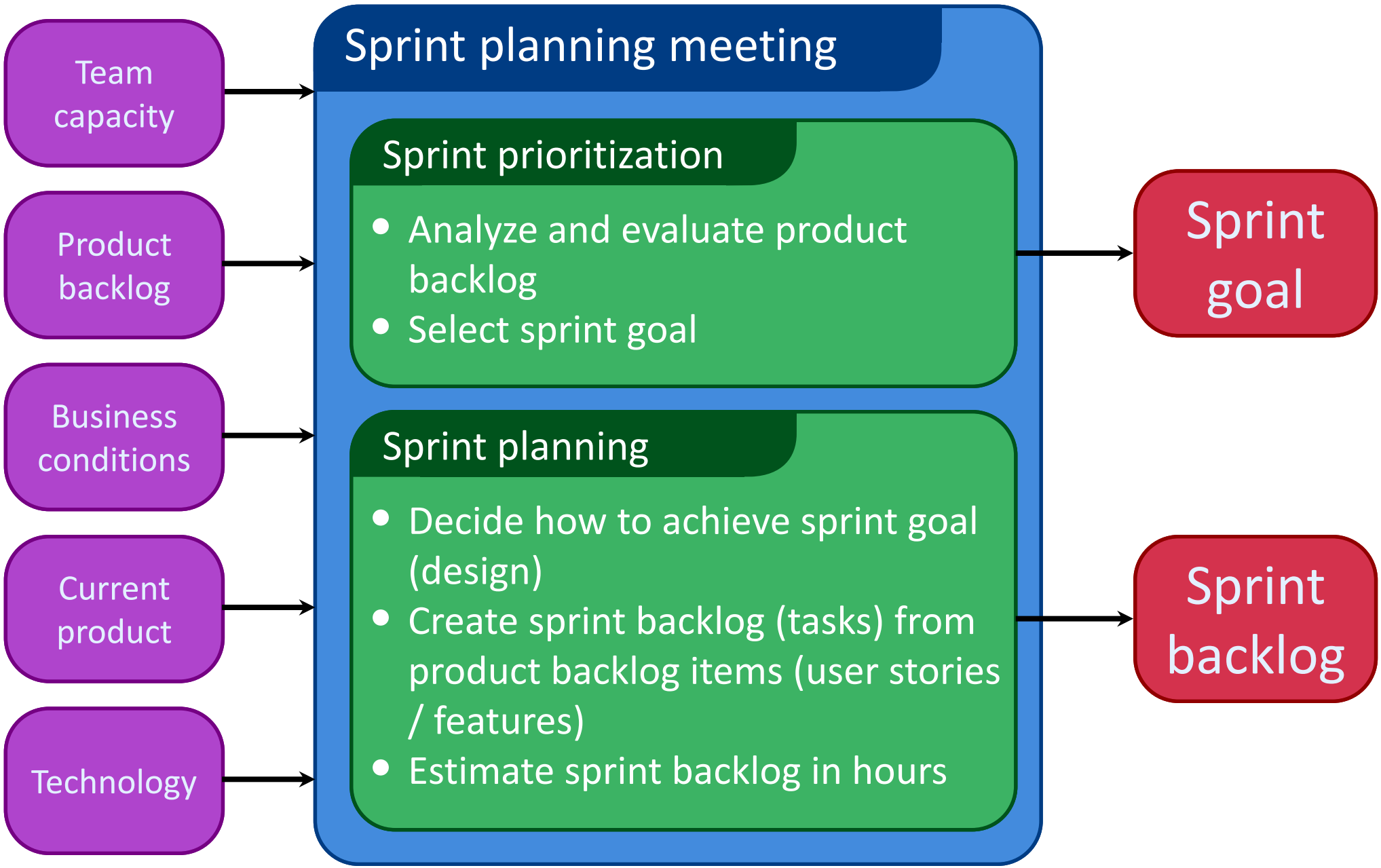


Overall Scrum Process

Artifact 1: Product backlog
A **totally ordered** list of
- user needs
- technical issues
- technical debts

Artifact 2: Sprint backlog
A **collection** of tasks

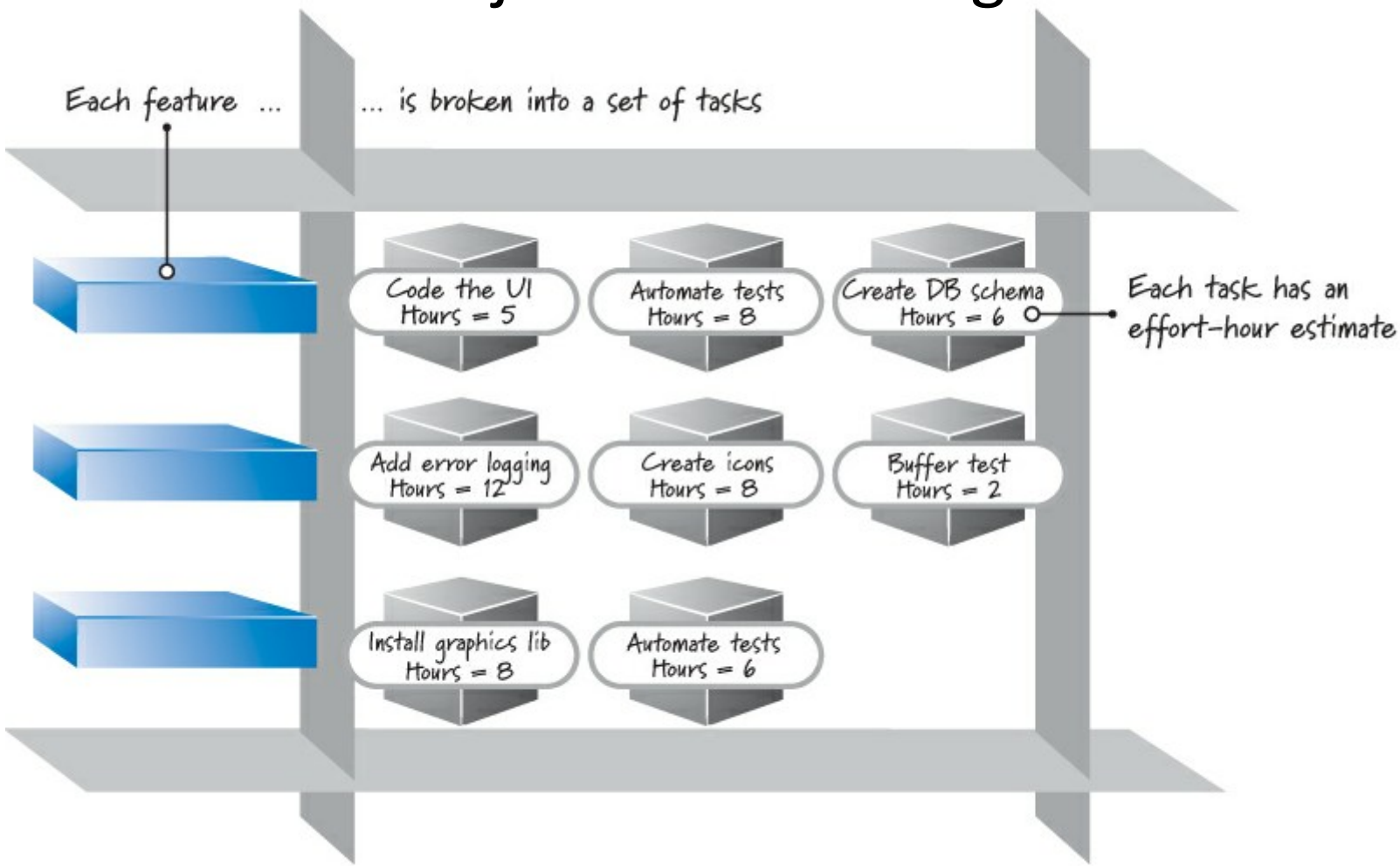



Team capacity
Product backlog
Business conditions
Current product
Technology
Sprint planning meeting
Sprint prioritization
Analyze and evaluate product backlog
Select sprint goal
Sprint planning
Decide how to achieve sprint goal (design)
Create sprint backlog (tasks) from product backlog items (user stories / features)
Estimate sprint backlog in hours
Sprint goal
Sprint backlog

Dibris

Sprint Backlog

just in time design



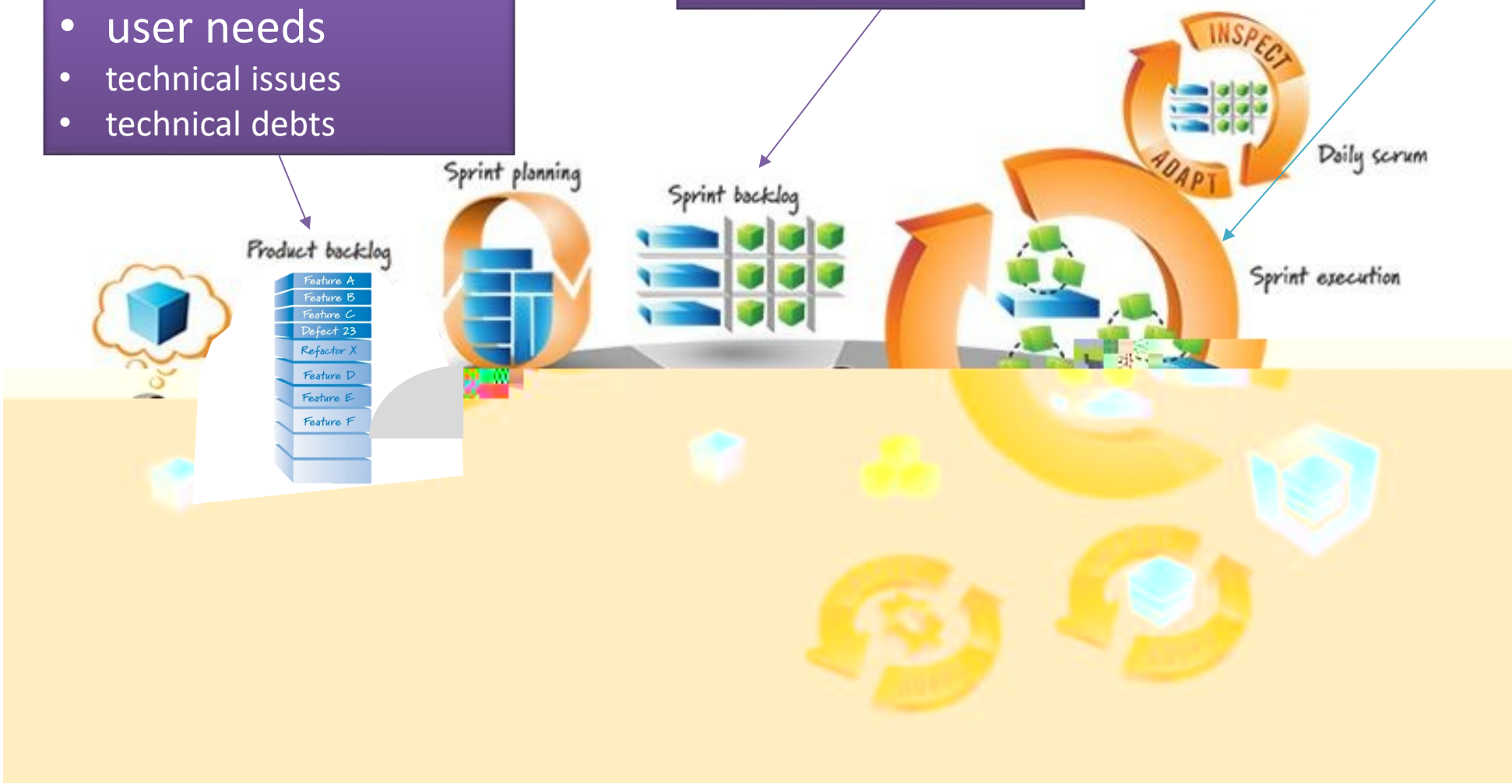
Looping on product backlog starting from top items till sprint capacity is reached

Overall Scrum Process

Artifact 1: Product backlog
A **totally ordered** list of
- user needs
- technical issues
- technical debts

Artifact 2: Sprint backlog
A **collection** of tasks

Good old style SW development
- no **prescribed** practices,
- we need some



Daily Scrum Meetings

One Meeting a Day Keeps the Other Meetings Away

- Why
  - to share the current status with the rest of the team
    - what did you do yesterday?
    - is anything in your way?
  - to jointly plan next day's work
    - what will you do today?
- When
  - every working day
    - it is the border between yesterday and today at whatever hour it takes place
  - no longer than 15 minutes
- Who
  - all team members, product owner, scrum master
  - any other stakeholder as silent spectator

Managing the sprint backlog

- Any team member can add, delete or change the sprint backlog
  - Nobody but team members can!
  - Work for the sprint emerges
- Individuals sign up for work of their own choosing
  - Work is never assigned

- Estimated work remaining is updated daily
  - If work is unclear, define a sprint backlog item with a larger amount of time and break it down later
  - Update work remaining as more becomes known

Tasks	Mon	Tues	Wed	Thurs	Fri
Code the user interface	8	4	8		
Code the middle tier	16	12	10	4	
Test the middle tier	8	12	16	11	8
Write online help	12				
Write the foo class	8	8	8	8	8
Add error logging			8	4	
Effort needed to complete	**52**	**40**	**50**	**27**	**16**

I will work with you

but we will need fresh data-sets

Yesterday I worked with Ben on task T666 but we did not finish it

we need help to understand the current architecture

Yesterday I replaced Ben on task T1024 and completed it with Anne. Then I started task T42 with Frank

Thanks, then in the morning we can help somebody on something else

no problems at all

Then let me start on T55, who wants to join me?

Today I'll work on T3 with Anna

Task T55 presents high business risks I would like to see it implemented soon



I will get them for you by 10 am

OK, then if it's ok for Anna, we will work on T11





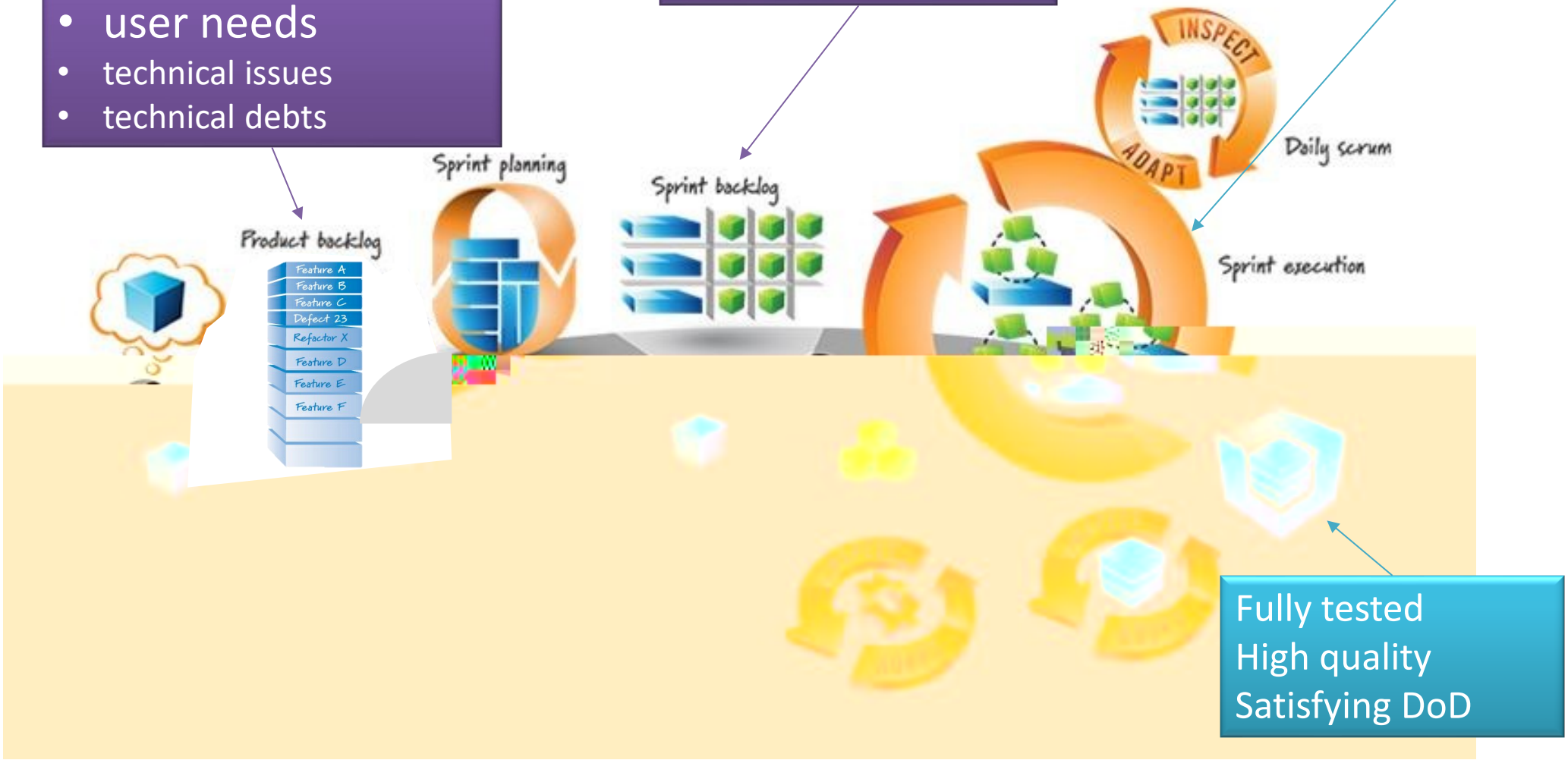
Dibris

Overall Scrum Process

Artifact 1: Product backlog
A **totally ordered** list of
- user needs
- technical issues
- technical debts

Artifact 2: Sprint backlog
A **collection** of tasks

Good old style SW development
- no **prescribed** practices,
- we need some



Fully tested
High quality
Satisfying DoD

The sprint review

- Inspection of the sprint product increment
  - demo of new features or underlying architecture
  - informal
    - 2-hour prep time rule
    - No slides
- Whole team *participates*
- Outcomes
  - Product owner approves/rejects
  - First feedback from stakeholders

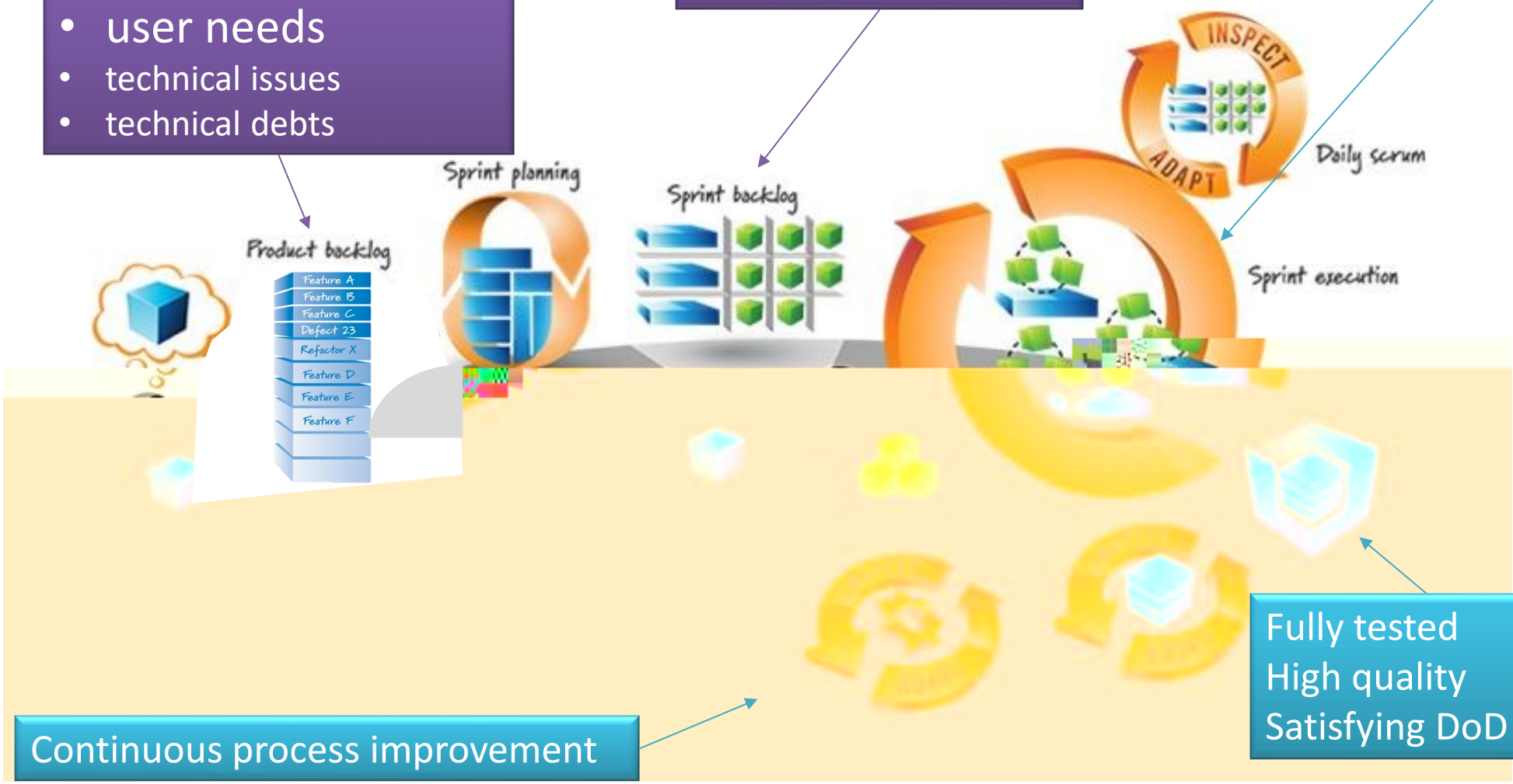


Overall Scrum Process

Artifact 1: Product backlog
A **totally ordered** list of
- user needs
- technical issues
- technical debts

Artifact 2: Sprint backlog
A **collection** of tasks

Good old style SW development
- no **prescribed** practices,
- we need some



Continuous process improvement

Fully tested
High quality
Satisfying DoD

Sprint retrospective

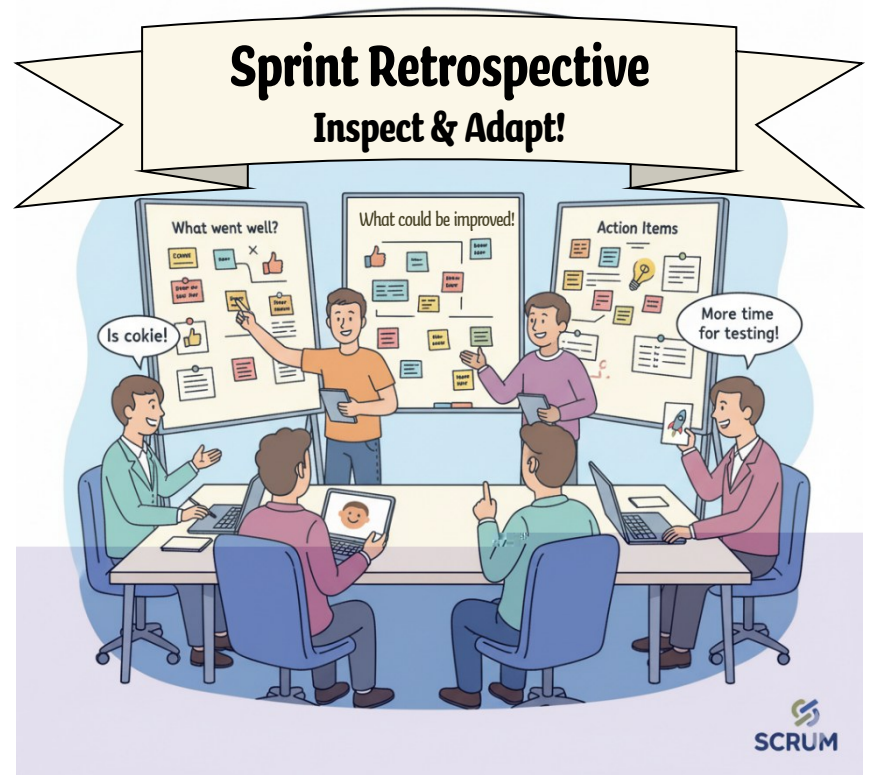
- Critique of development process
  - What worked
  - What did not work
  - New practices that could help
- Typically, 15–30 minutes
- Key roles *participate*
  - ScrumMaster
  - Product owner
  - Team
- Possibly others
  - Customer/Stakeholders
  - [Process] Consultants
  - “Apprentices”

Keep it!

Stop it!

Replacement

Give it a try!

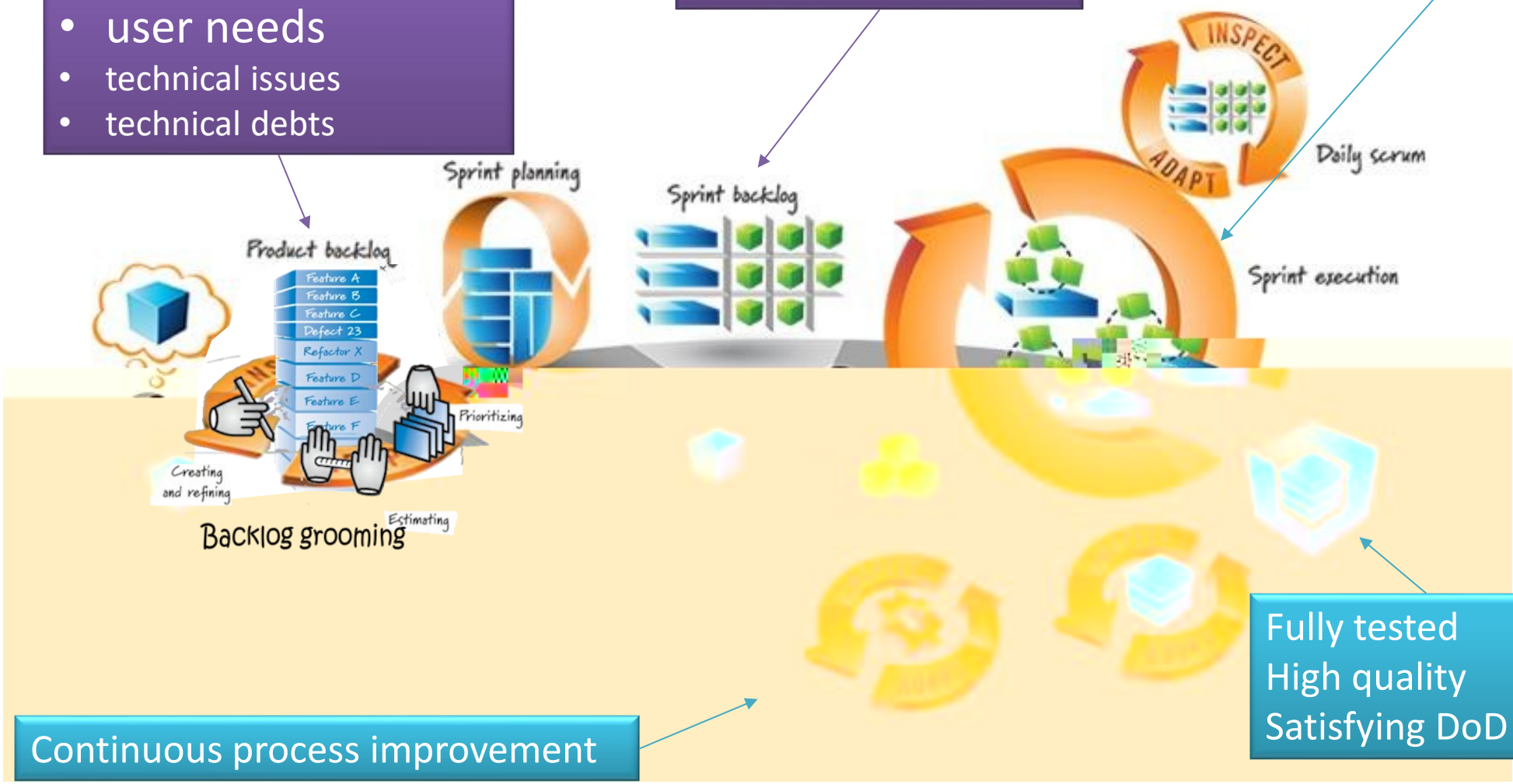


Overall Scrum Process

Artifact 1: Product backlog
A **totally ordered** list of
- user needs
- technical issues
- technical debts

Artifact 2: Sprint backlog
A **collection** of tasks

Good old style SW development
- no **prescribed** practices,
- we need some

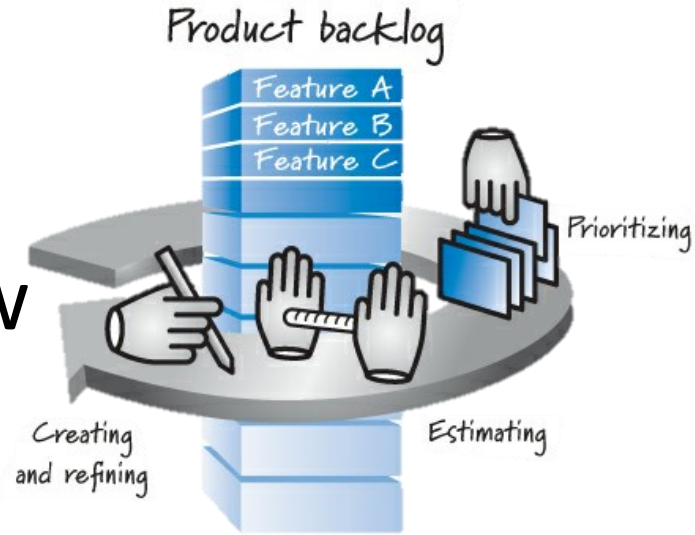


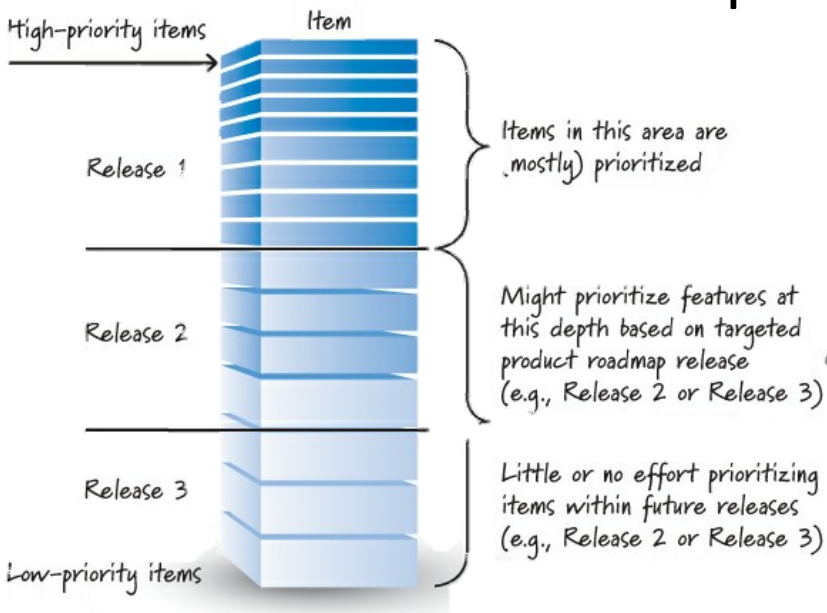
Continuous process improvement

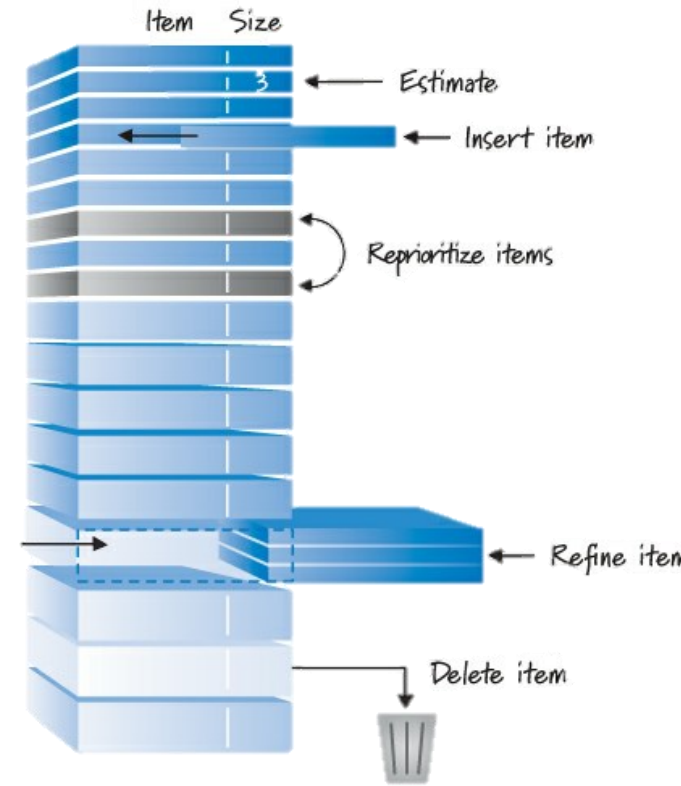
Fully tested
High quality
Satisfying DoD

Backlog grooming/refinement

- Last *ceremony* added
- Not clear position in the work flow
  - within sprint planning
  - within sprint review
  - standalone meeting
    - somewhere toward end of sprint







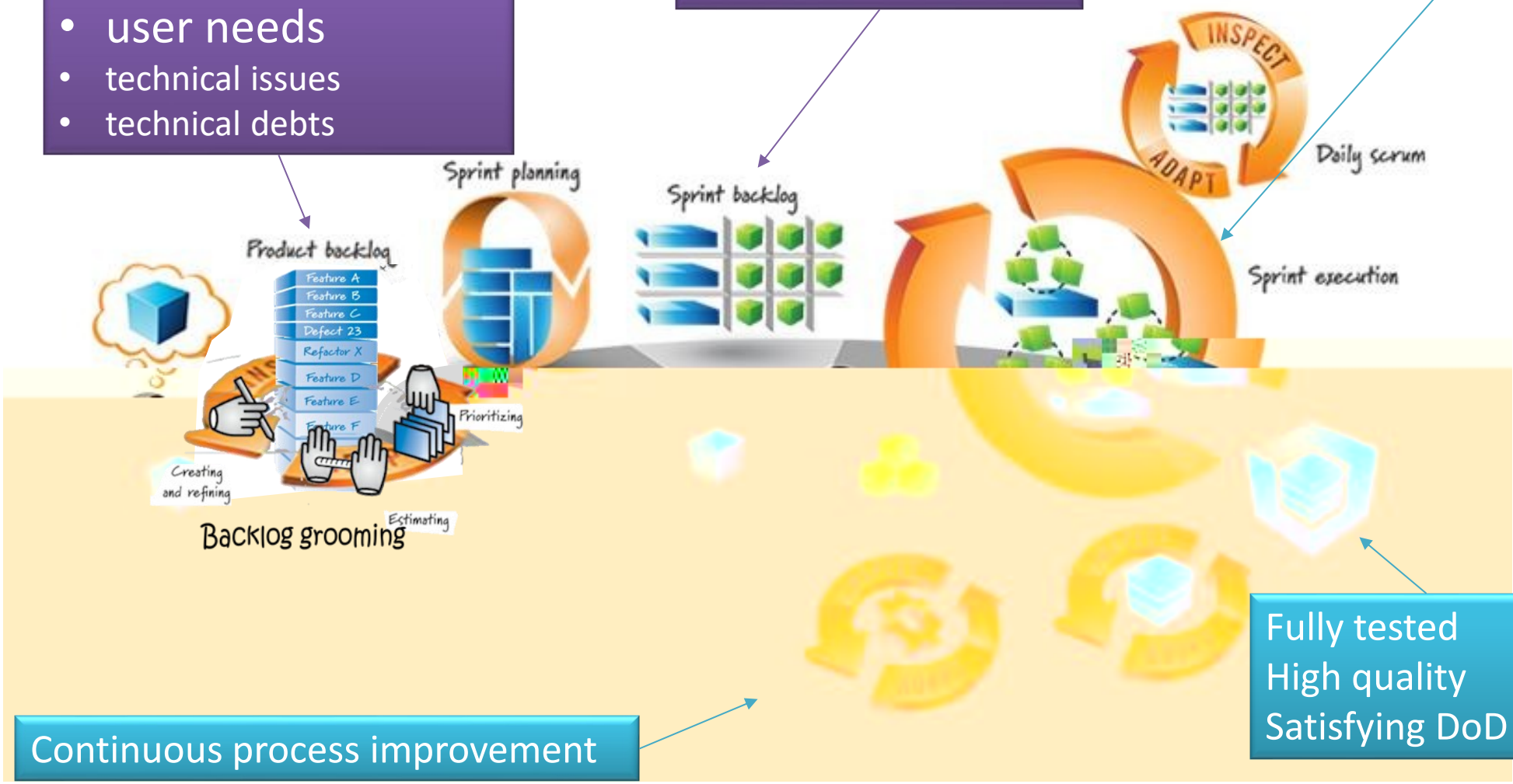
Dibris

Overall Scrum Process

Artifact 1: Product backlog
A **totally ordered** list of
- user needs
- technical issues
- technical debts

Artifact 2: Sprint backlog
A **collection** of tasks

Good old style SW development
- no **prescribed** practices,
- we need some



Continuous process improvement

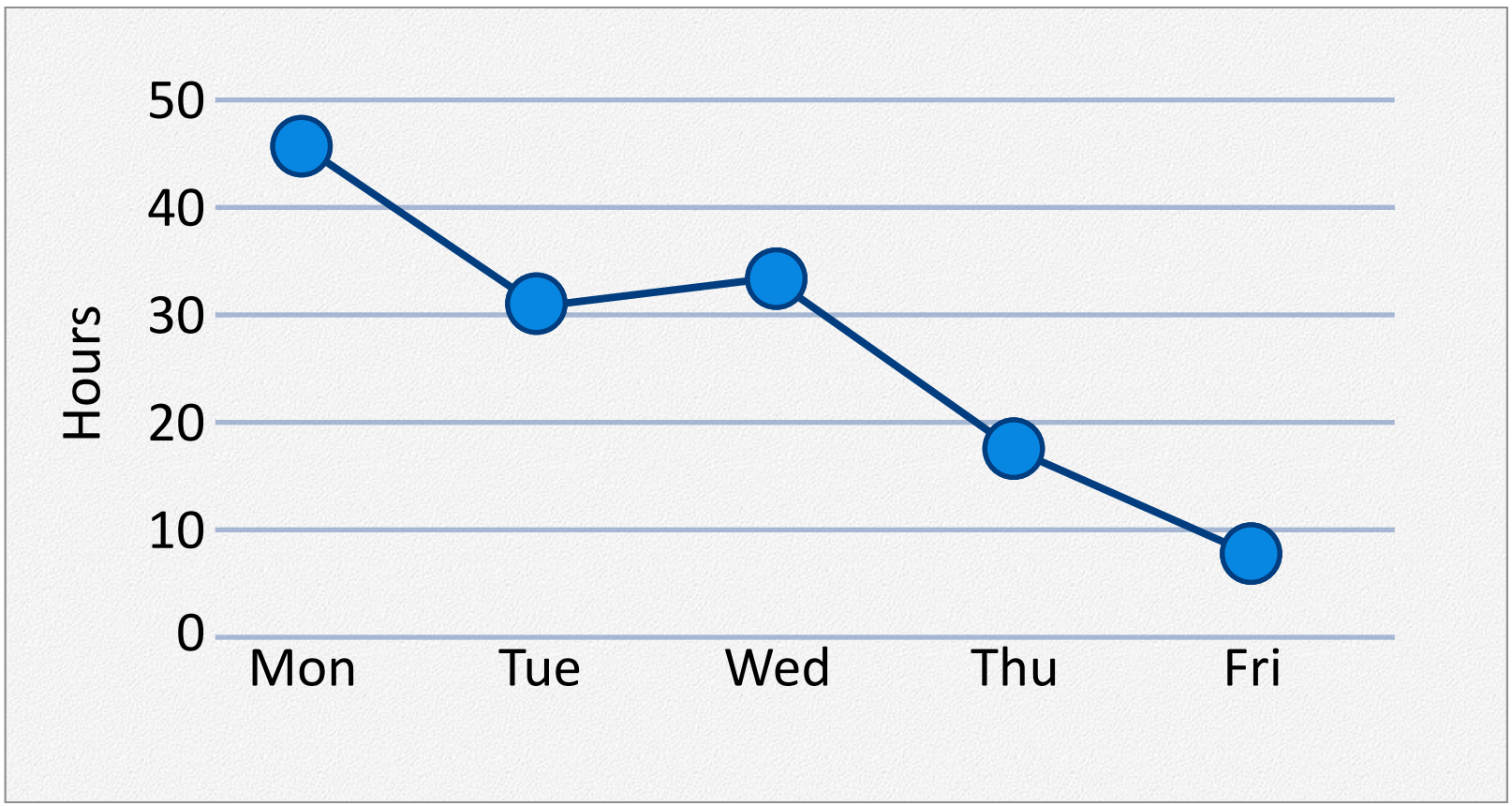
Fully tested
High quality
Satisfying DoD

All activities are time boxed

Dibris

A Sprint Burn Down Chart

Tasks	Mon	Tues	Wed	Thur	Fri
Code the user interface	8	4	8		
Code the middle tier	16	12	10	7	
Test the middle tier	8	16	16	11	8
Write online help	12				
	44	32	34	18	8



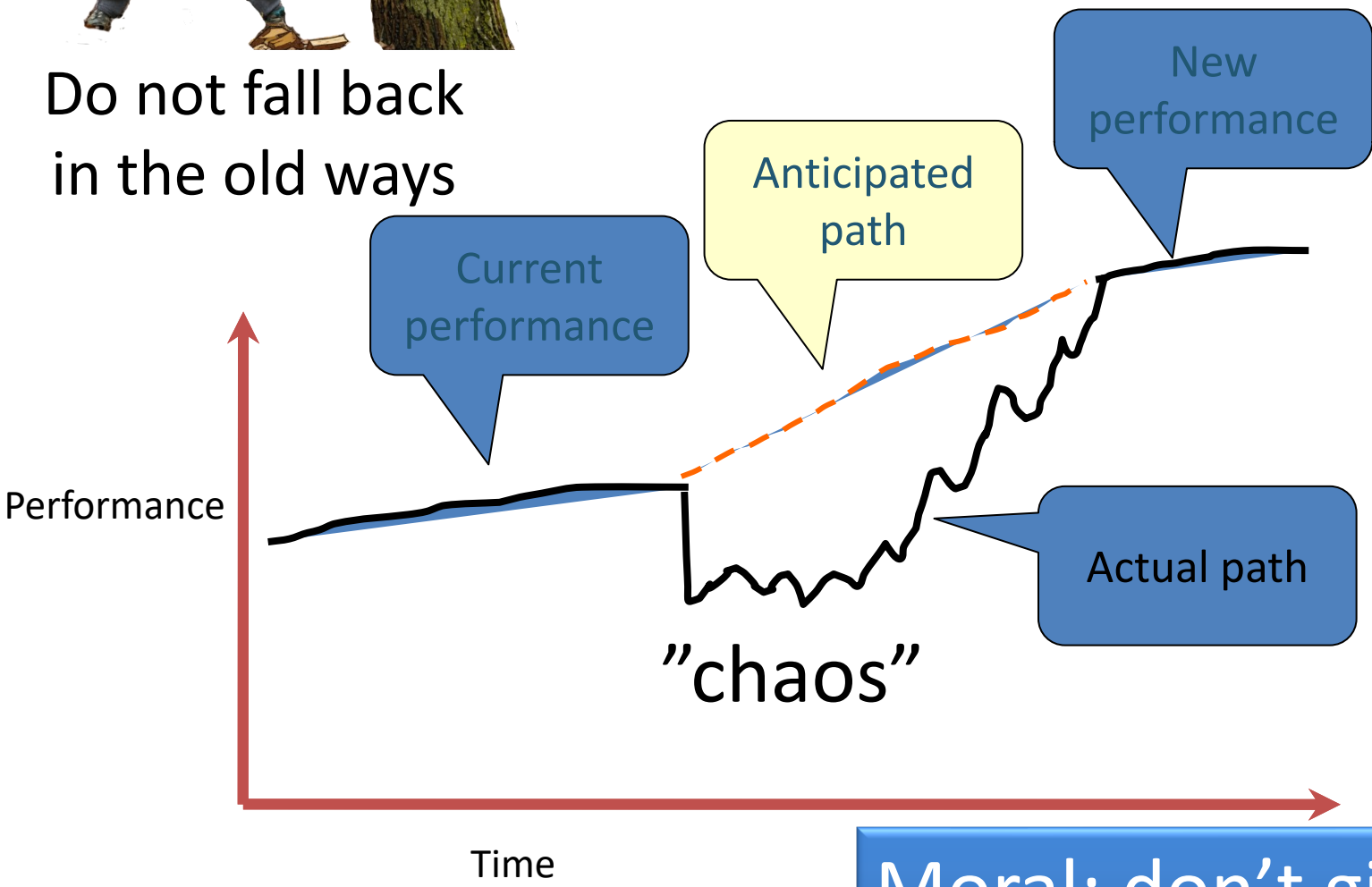
Dibris



Satir model

”the middle always looks like failure”

Do not fall back
in the old ways



Moral: don’t give up to the first difficulties

Dibris

OK, Let's do it!

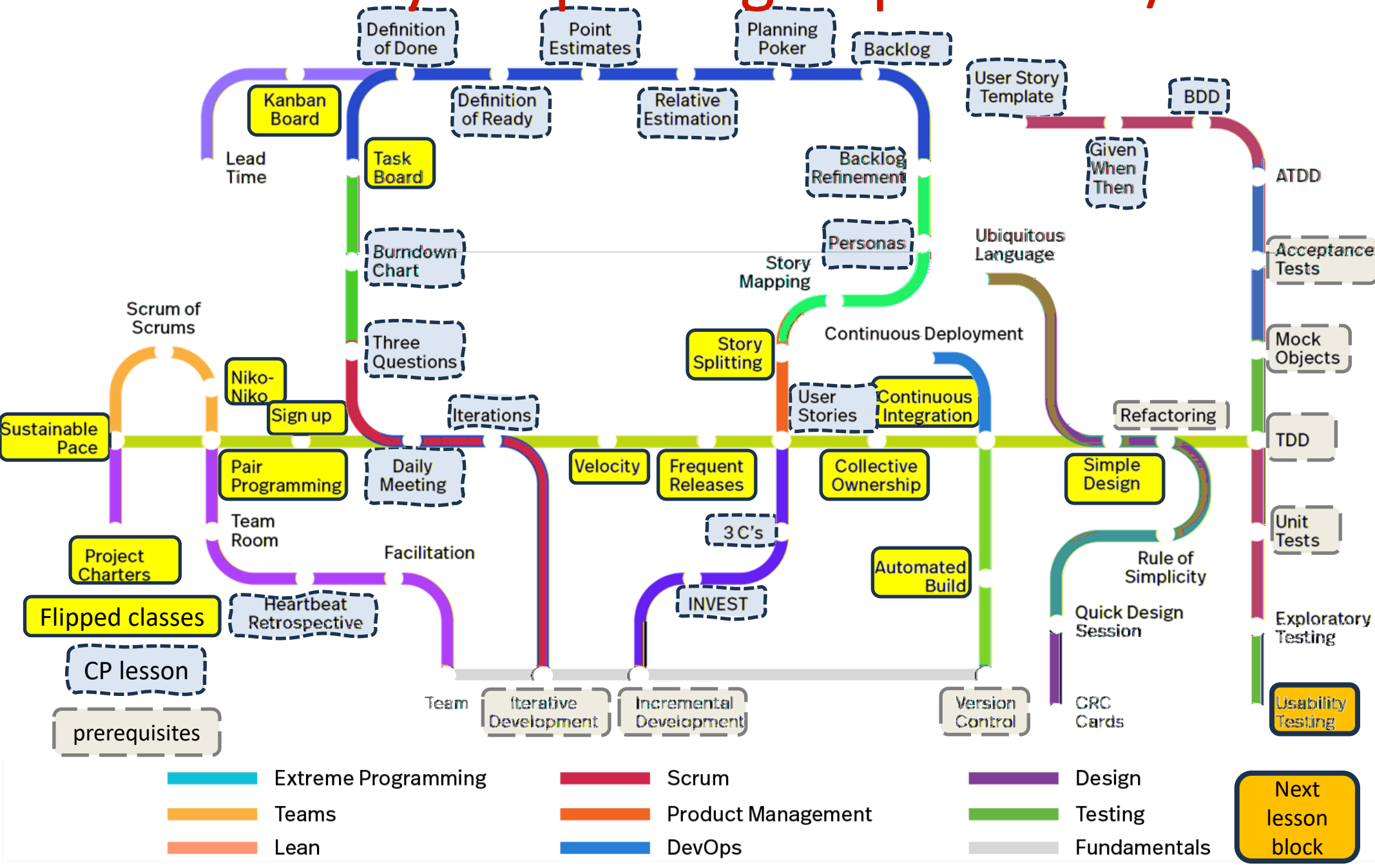
- First step to apply Scrum is….



- Problem
  Scrum is a high-level method
  - principles
  - overall process and milestones/ceremonies
- Missing low-level details
  - day-to-day work practices
    - just a bit here and there…daily meeting's 3 questions
  - choices of techniques and tools
    - not only for coding
    - just a bit here and there…
      burn down charts

Flipped lessons
Practices and techniques/tools
which we *could* adopt

https://agilealliance.org/agile101/subway-map-to-agile-practices/



Interactions
User Interfaces

Capstone Project

Maura Cerioli

User Interaction ⇔ User Interface

For most users Interface ≅ System

- ✓features
- ✓ease of use
- ×performances
  - besides interface responsiveness
- ×security/safety
  - besides the obvious

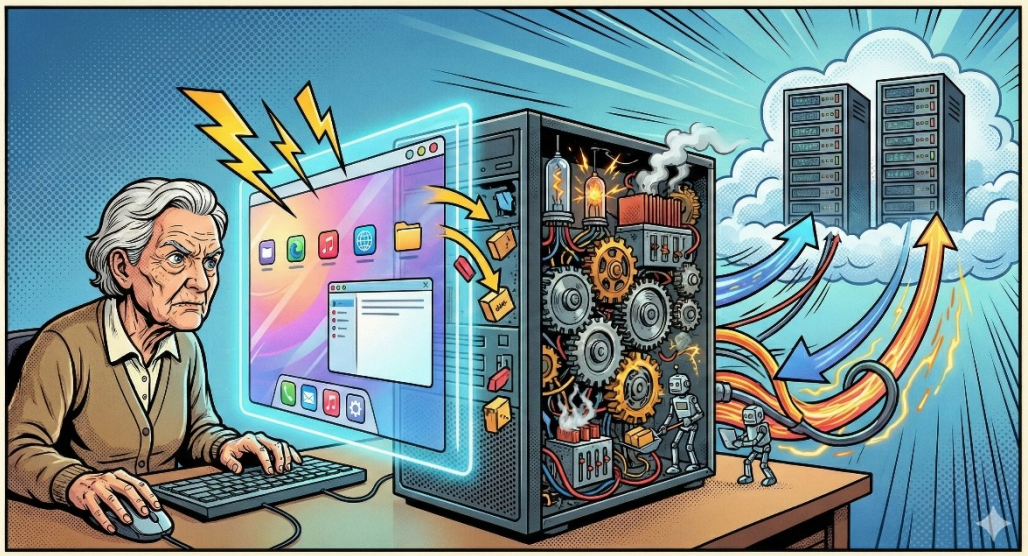
Improving interface

dramatic improvement of perceived quality

User



Interface determines user interactions





Limited by available components

Flexible and easier to creatively design



Dibris

Usability ⇔ Accessibility ⇔ User Experience



Usability

ease of use + learnability for specified users



Accessibility

availability to as many people as possible

User Experience
≅
usability + feelings



User Experience

A person's perceptions and responses that result from the use or anticipated use of a product, system or service

(also short for [good] user experience design)

Dibris

Usability

One or more metric for each facet
Categories for testing/measuring usability
Test = user observation

Facets

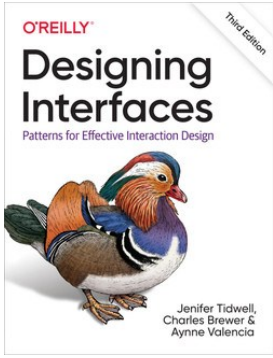
- Learnability
- Efficiency
- Memorability
- Errors
- Satisfaction

Get the *right* set of user
Sets them some tasks
Measure results

- # successes
- # errors
- # time to completion
- ….

Jakob Nielsen & Ben Shneiderman

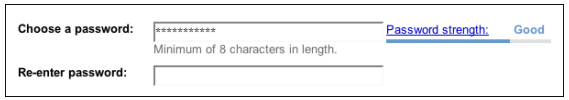
Interaction Patterns



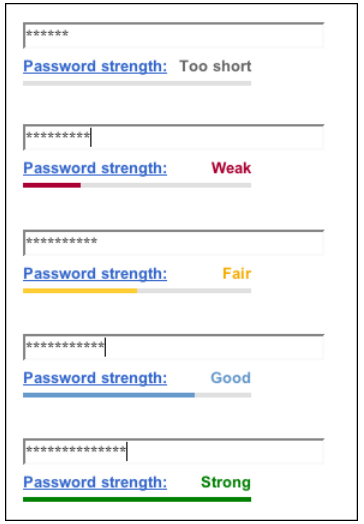
Many books/sites with catalogues

- sometimes listing design concepts that are not exactly design patterns
- e.g.,
  - https://ui-patterns.com/patterns
  - https://uxplaybook.org/articles/essential-interaction-design-patterns-and-techniques
  - https://www.smashingmagazine.com/2009/06/10-ui-design-patterns-you-should-be-paying-attention-to/
- find out more on your own
  - it might be your secret weapon for the project
- many examples seen web surfing
  - still useful to bring them to conscious level

Password Strength Meter



- What: give the user immediate feedback on the validity and strength of a new password while it is being typed
- Use when: the UI asks the user to choose a new password
- Why: strong passwords protect both users and site
  - weak passwords ought to be disallowed
  - immediate feedback
    - instantly, not after form submission
- How: while user typing/as focus leaves the text field, show an estimate of the password strength beside the text field.
  - at minimum, text /graphic label indicating a weak, medium, or strong password + wording to describe weakness, if any
  - if possible, additional specific advice on how to produce a strong password
  - never actually show a password/make suggestions of alternative passwords
- Examples

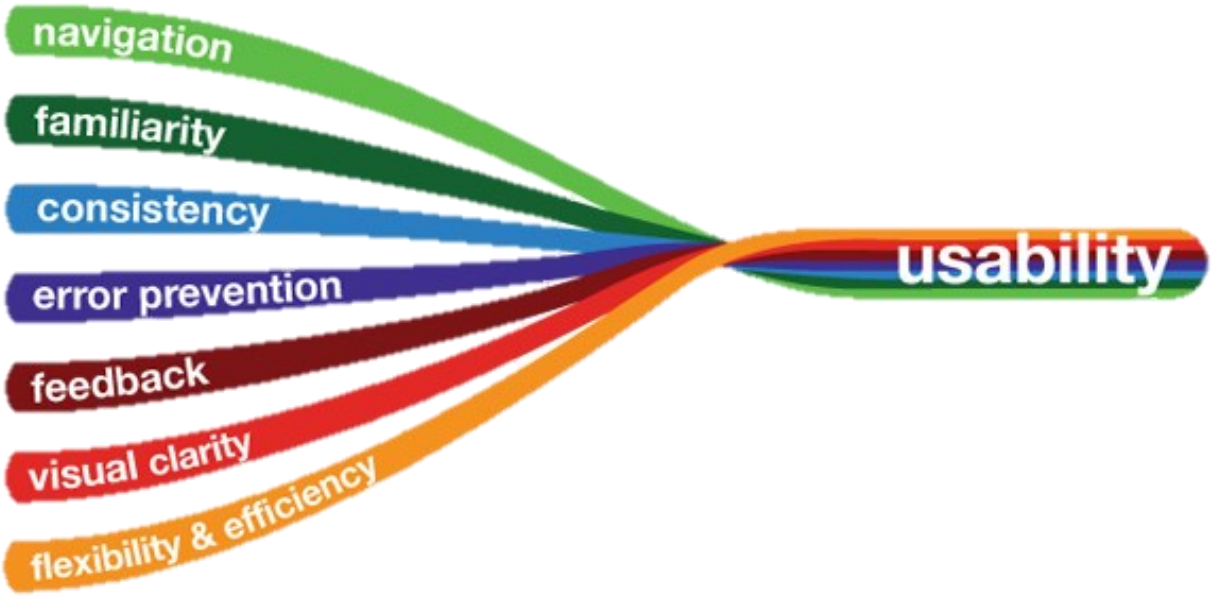




Dibris

Usable…but HOW?

- Several lists of
  - tips
  - areas of concern
- Based on results from cognitive science
  - human physiology
  - human standard behaviour



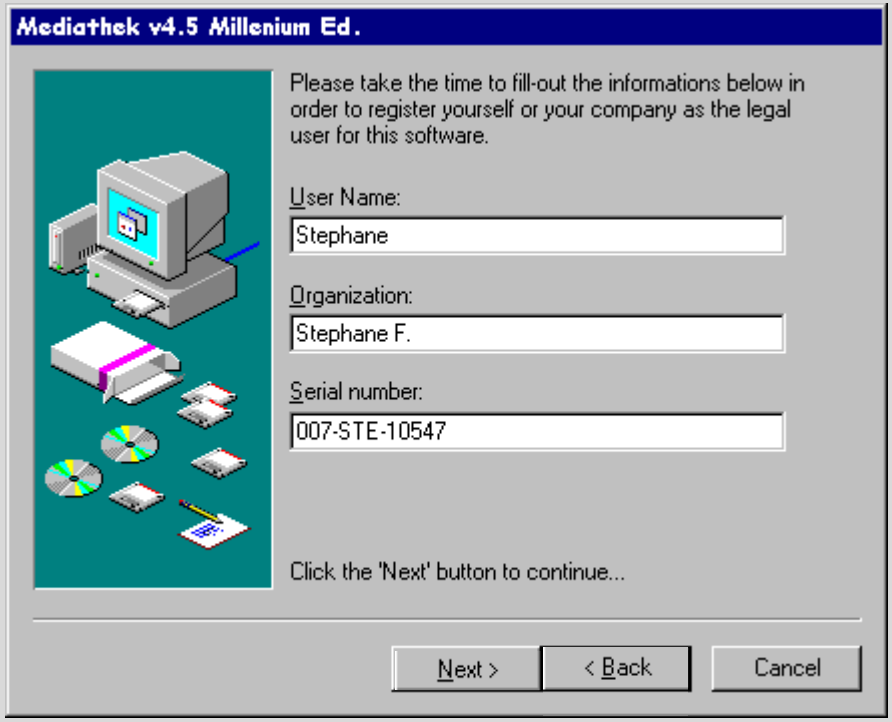
Facts about Human Perception & Cognition

1. We perceive what we expect
2. Our vision is optimized to see structure
3. Our color vision is limited
4. Our peripheral vision is poor
5. Our attention is limited
6. Our memory is imperfect
7. Recognition is easy; recall is hard
8. We think mostly about our tasks, not our tools
9. We seek and use structure
10. Inductive reasoning is easy; deduction is hard
11. Human thought-cycle: goal, execute, evaluate
12. We have real-time requirements

Jeff Johnson

Facts about Human Perception & Cognition

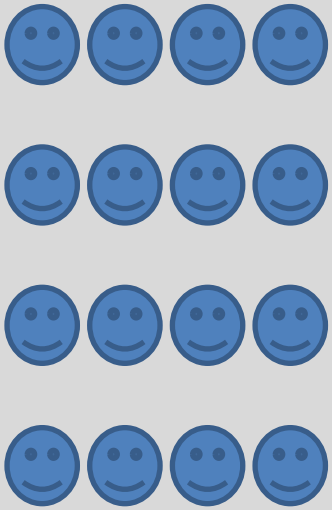
1. We perceive what we expect

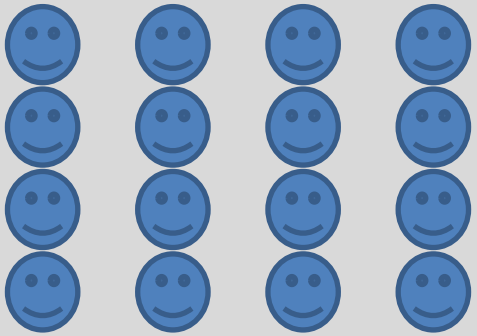


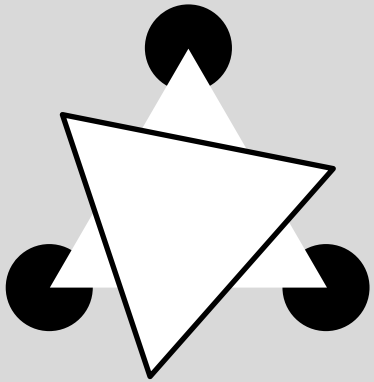
Dibris

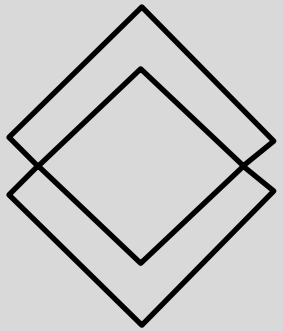
Facts about Human Perception & Cognition

2. Our vision is optimized to see structure





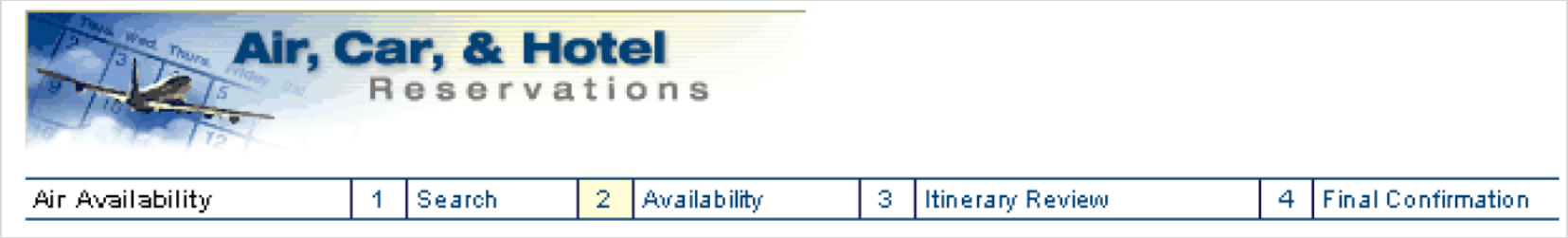




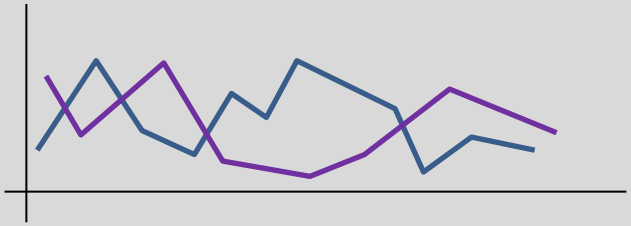
Facts about Human Perception & Cognition

3. Our color vision is limited

```
30
31    test 'should not multiple create posts' do
32      assert_difference('Post.count') do
33        post :create, post: @updated_post
34      end
35      assert_no_difference('Post.count') do
36        post :create, post: @updated_post
37      end
38    end
```



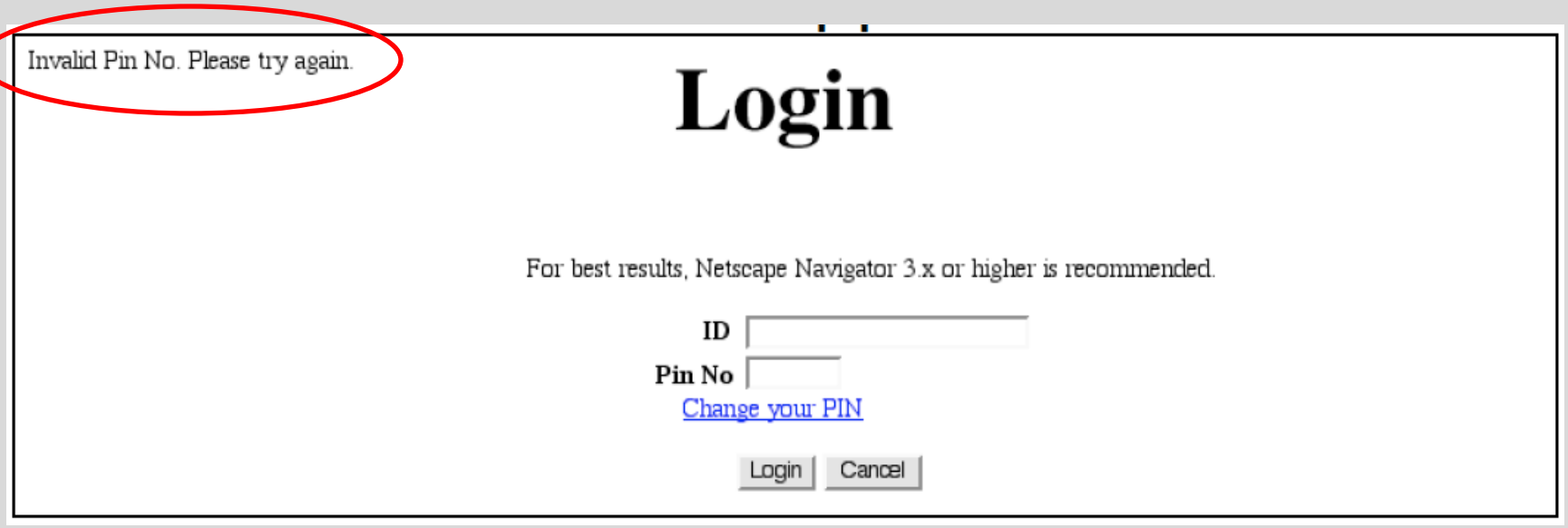
colour blindness: ~8% male, ~0.5% female



Dibris

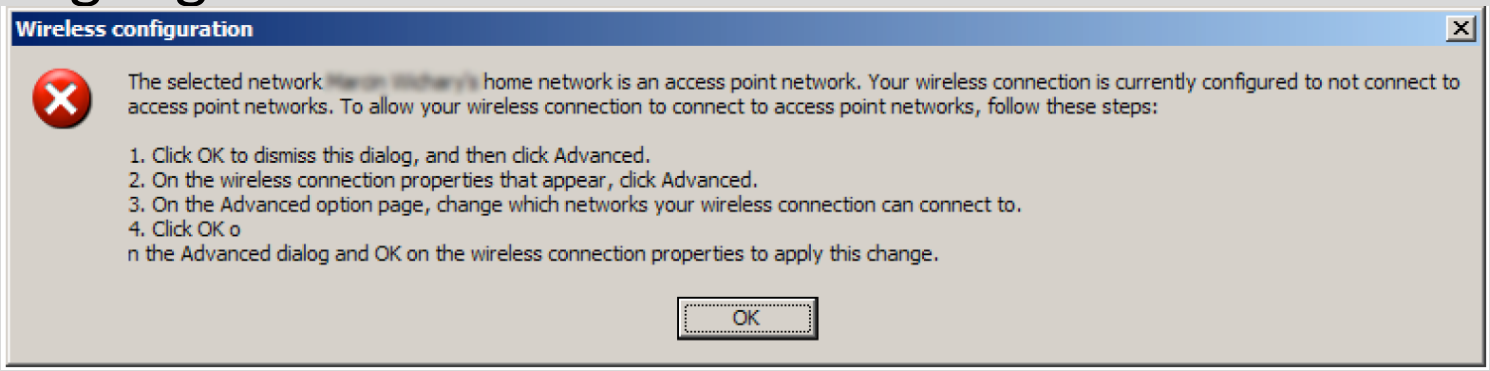
Facts about Human Perception & Cognition

4. Our peripheral vision is poor



Facts about Human Perception & Cognition

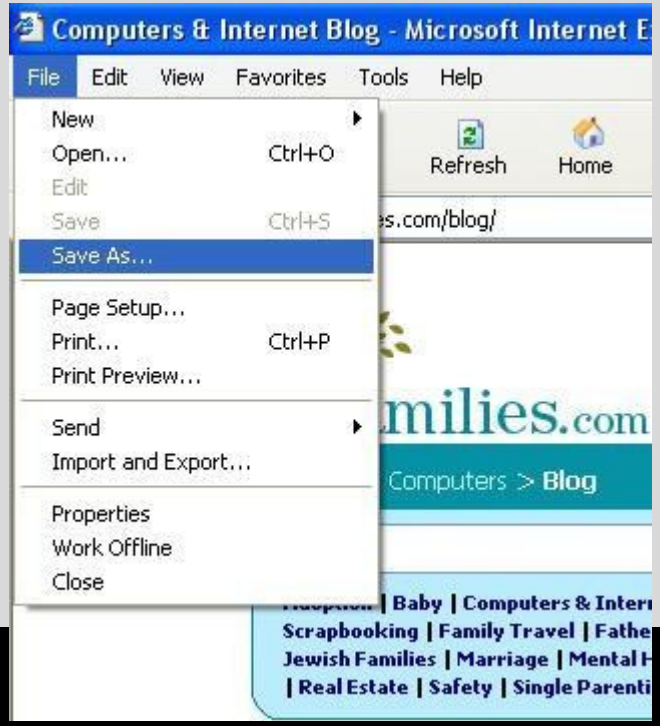
5. Our attention is limited
6. Our memory is imperfect
   Short-term memory (STM)
   - Represents conscious mind
     - Attention: what we're attending to NOW
   - Holds 3-7 unrelated items
   - Goals, numbers, words, images, other info
   - New items can "bump" old ones out
   - Easy to forget goals or info



Dibris

Facts about Human Perception & Cognition

7. Recognition is easy; recall is hard

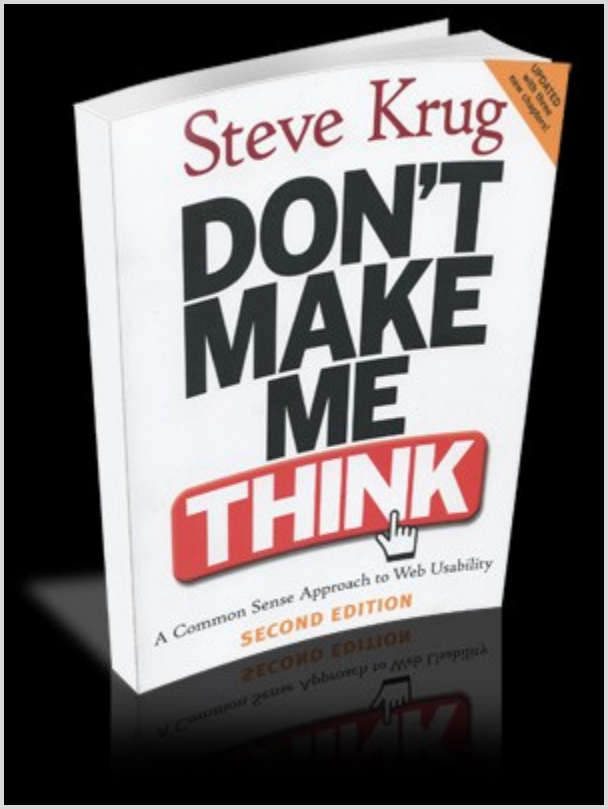




Dibris

Facts about Human Perception & Cognition

8. We think mostly about our tasks, not our tools



Dibris

Facts about Human Perception & Cognition

9. We seek and use structure

Create a Clear Visual Hierarchy
Organize and prioritize the contents of a page by using size, prominence, and content relationships. Let's look at these relationships more closely.
The more important a headline is, the larger its font size should be.
Big bold headlines help to grab the user's attention as they scan the Web page. The more important the headline or content, the higher up the page it should be placed.
The most important or popular content should always be positioned prominently near the top of the page, so users can view it without having to scroll too far. Group similar content types by displaying the content in a similar visual style, or in a clearly defined area.

Dibris

Create a Clear Visual Hierarchy

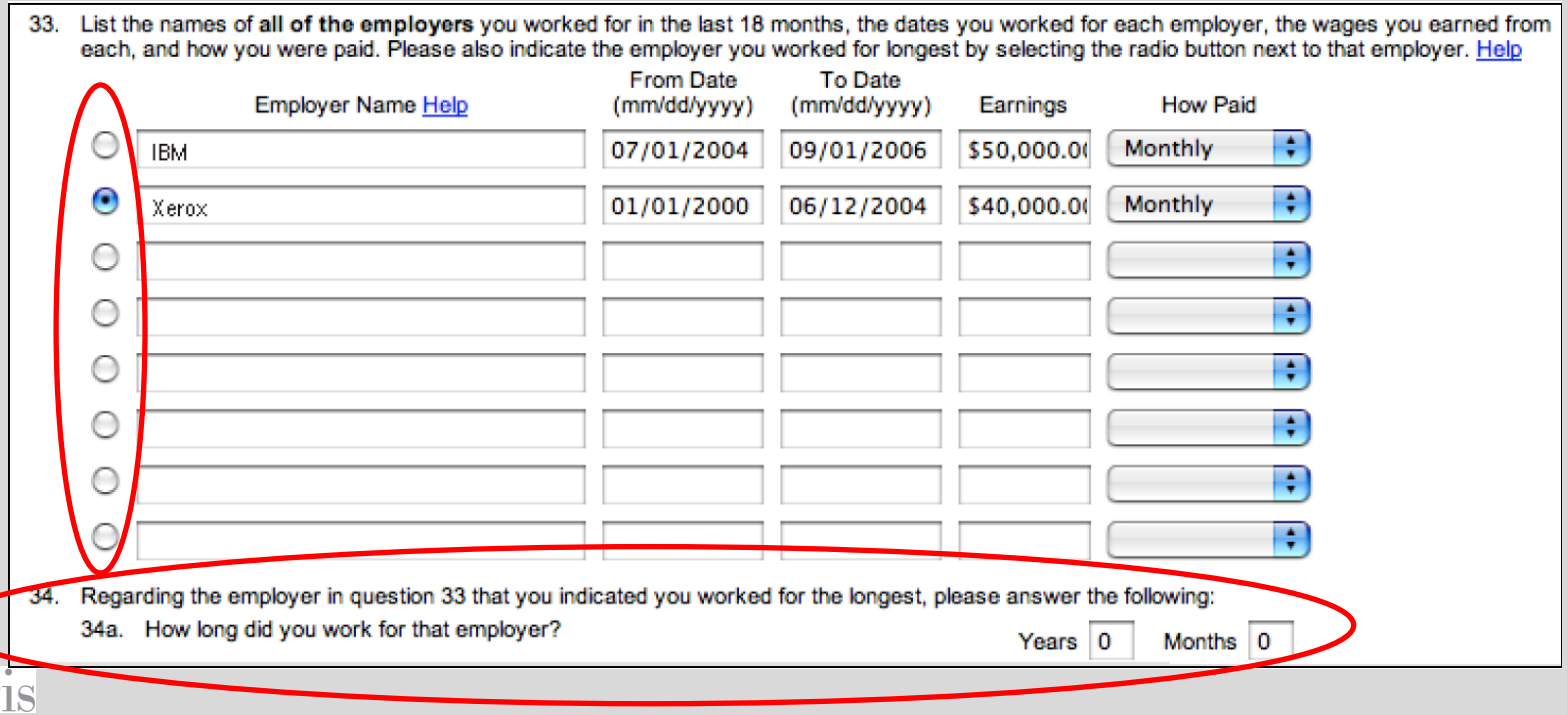
Organize and prioritize the contents of a page by using size, prominence, and content relationships.

Let's look at these relationships more closely:

- **Size**. The more important a headline is, the larger its font size should be. Big bold headlines help to grab the user's attention as they scan the Web page.
- **Prominence**. The more important the headline or content, the higher up the page it should be placed. The most important or popular content should always be positioned prominently near the top of the page, so users can view it without having to scroll too far.
- **Content Relationships.** Group similar content types by displaying the content in a similar visual style, or in a clearly defined area.

Facts about Human Perception & Cognition

10. Inductive reasoning is easy; deduction is hard
    = learning from experience is natural, making calculations is not
    ⇒ don't make users deduce/compute unless strictly necessary



33. List the names of **all of the employers** you worked for in the last 18 months, the dates you worked for each employer, the wages you earned from each, and how you were paid. Please also indicate the employer you worked for longest by selecting the radio button next to that employer. Help

Employer Name Help	From Date (mm/dd/yyyy)	To Date (mm/dd/yyyy)	Earnings	How Paid
IBM	07/01/2004	09/01/2006	$50,000.0(	Monthly
Xerox	01/01/2000	06/12/2004	$40,000.0(	Monthly

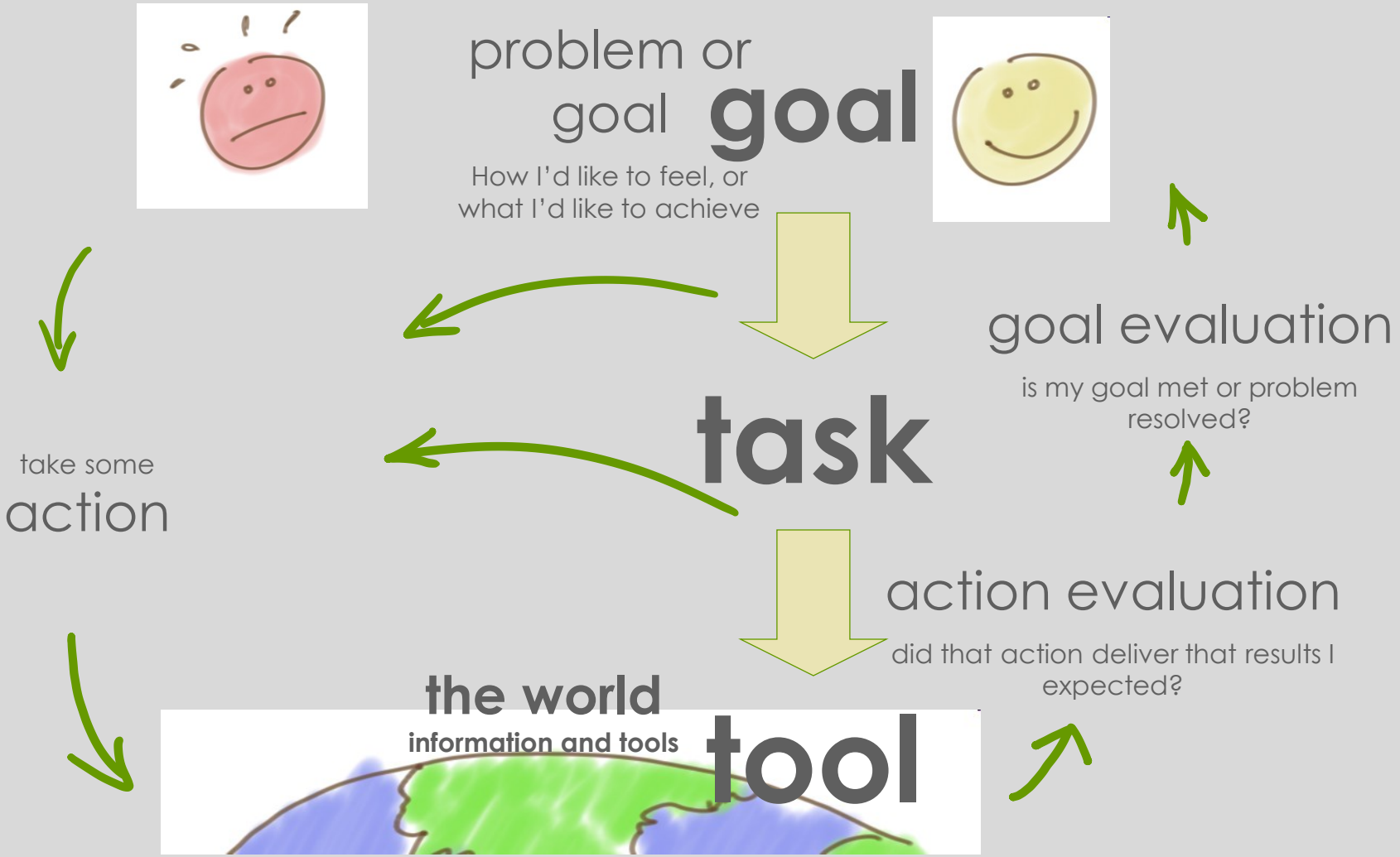
34. Regarding the employer in question 33 that you indicated you worked for the longest, please answer the following:

34a. How long did you work for that employer? Years 0 Months 0

Dibris

Facts about Human Perception & Cognition

11. Human thought-cycle: goal, execute, evaluate (Norman's model)



Dibris

Facts about Human Perception & Cognition

12. We have real-time requirements
    - 0.1 Second - Governs human
        - Perceptual-motor time-loop
        - Perception of cause-effect
        - Perceptual fusion
    - 1 Second - Governs human
        - Turn-taking in conversation
        - Minimum reaction time for unexpected events
    - 10 Seconds - Governs human
        - Unbroken attention to a task
        - “Unit task”







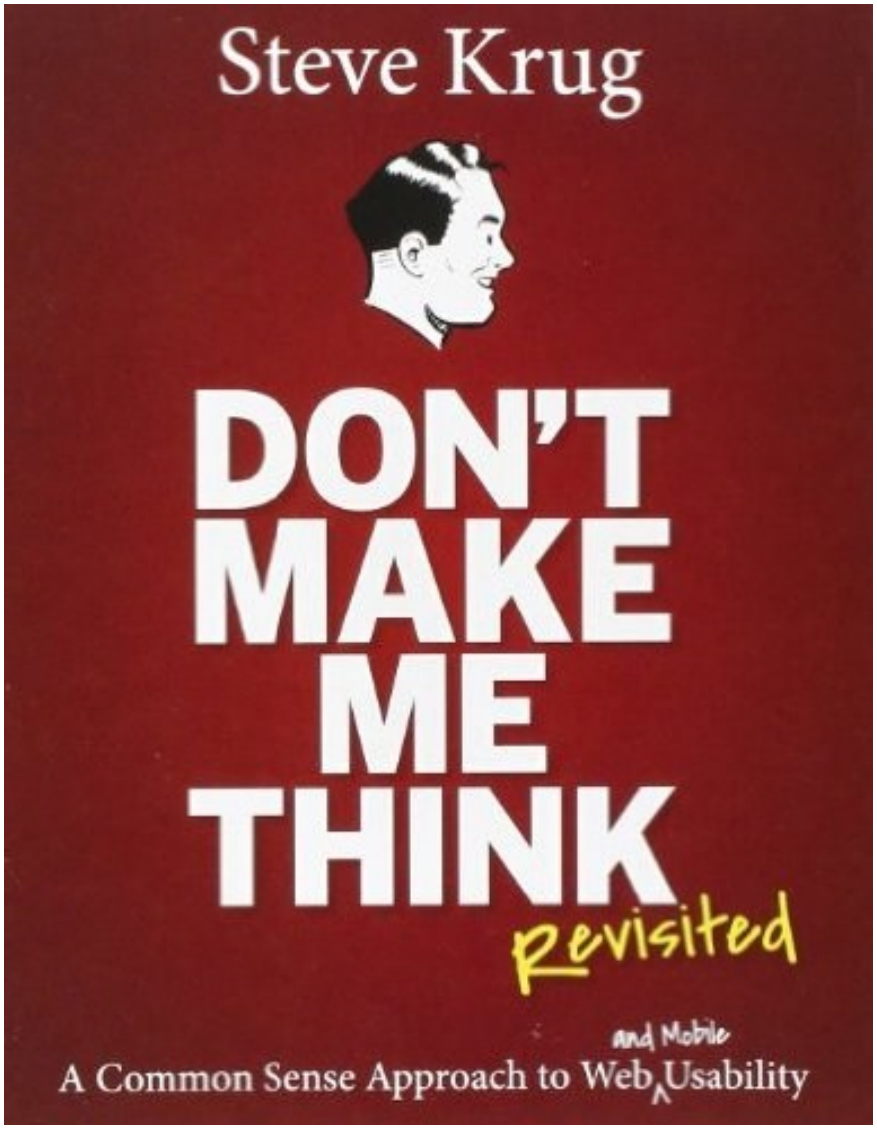




Dibris

User interface design principles

Principle	Description
User familiarity	The interface should use terms and concepts which are drawn from the experience of the people who will make most use of the system.
Consistency	The interface should be consistent in that, wherever possible, comparable operations should be activated in the same way.
Minimal surprise	Users should never be surprised by the behaviour of a system.
Recoverability	The interface should include mechanisms to allow users to recover from errors.
User guidance	The interface should provide meaningful feedback when errors occur and provide context-sensitive user help facilities.

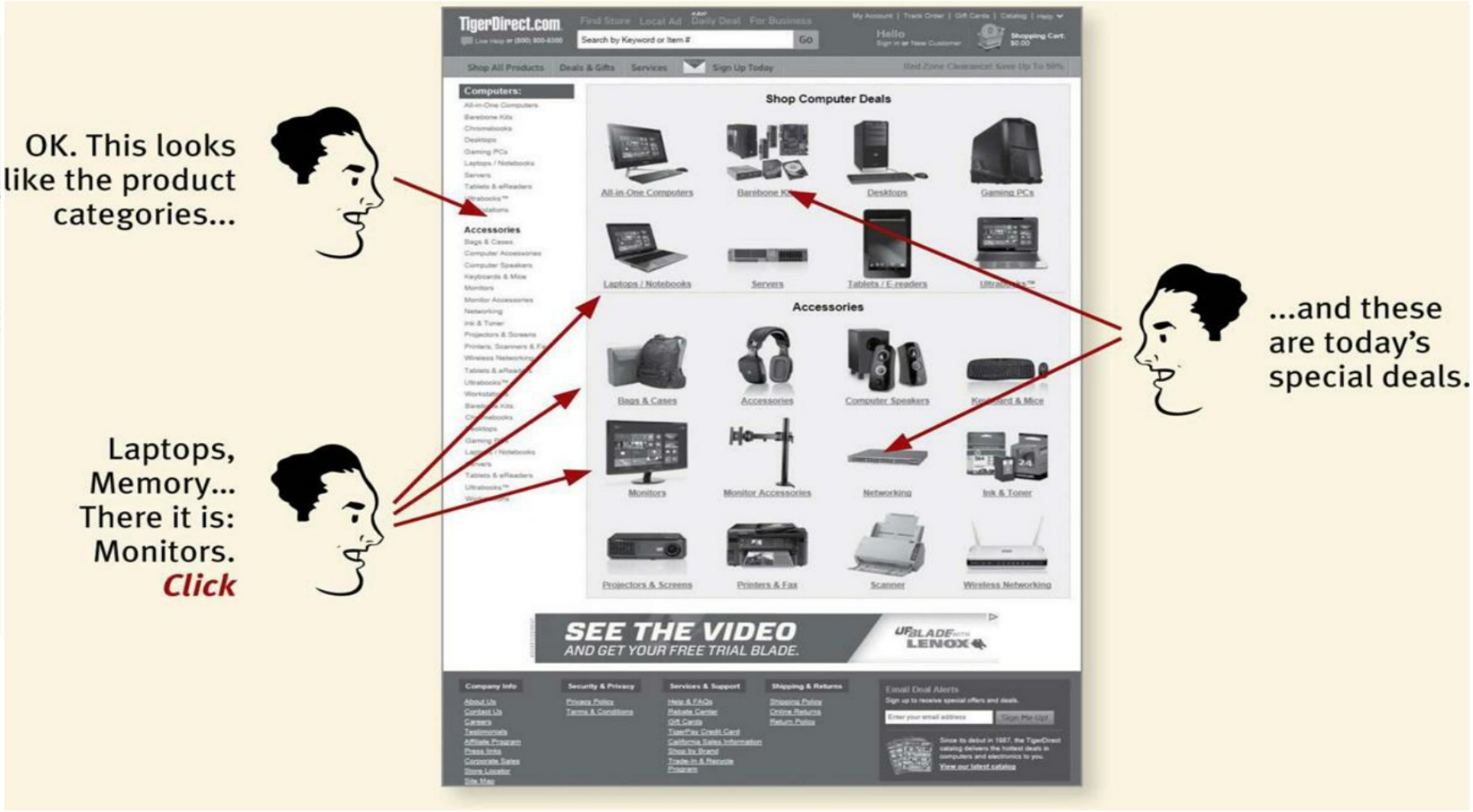


Steve Krug - New Riders Pub
2nd revised edition
26th of August 2005

Steve Krug - New Riders Pub
3rd revised edition
24th of December 2013

Addresses also Mobile Apps

Think? in Which Sense?

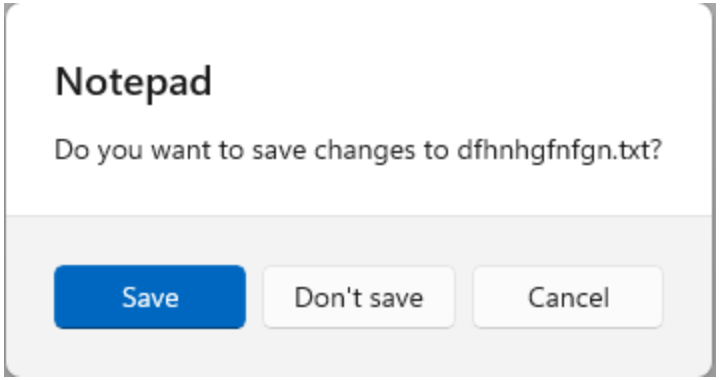


UX designer goal: getting rid of those ???

Dibris

Other examples







Making every detail self-evident is like having good lighting in a store it just makes everything seem better.

Dibris

A few (non sw) Design Disasters





userinyerface.com





Dibris

Designers' Expectations vs. Reality

WHAT WE DESIGN FOR...

Read

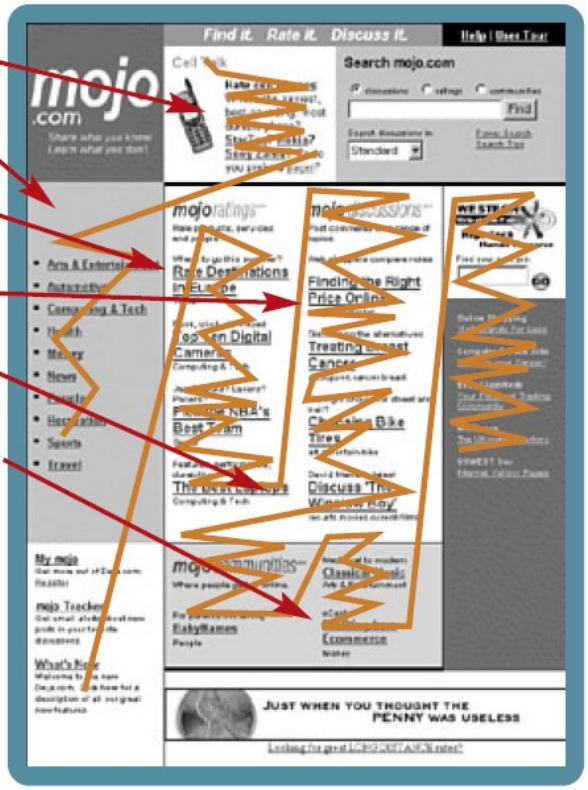
Read

Read

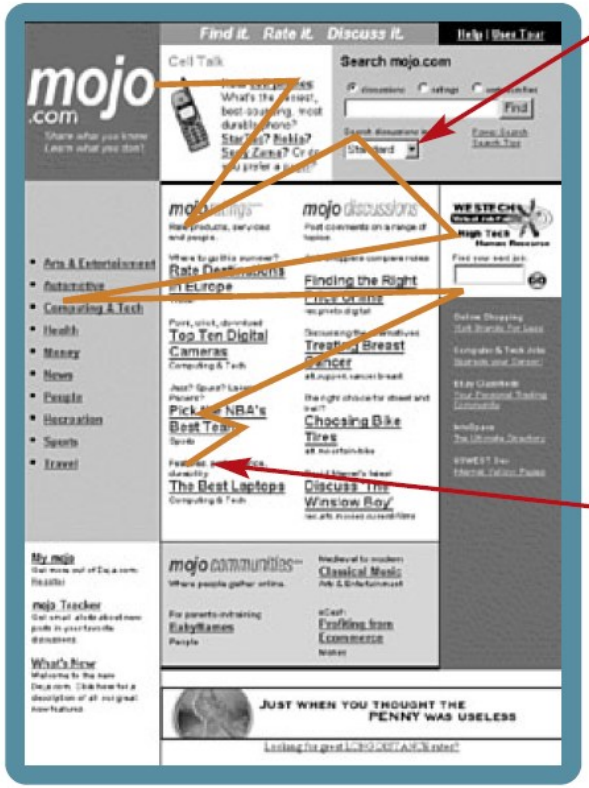
Read

[Pause for reflection]

Finally, click on a carefully chosen link



THE REALITY...



Look around feverishly for anything that

a) is interesting, or vaguely resembles what you're looking for, and

b) is clickable.

As soon as you find a halfway-decent match, click.

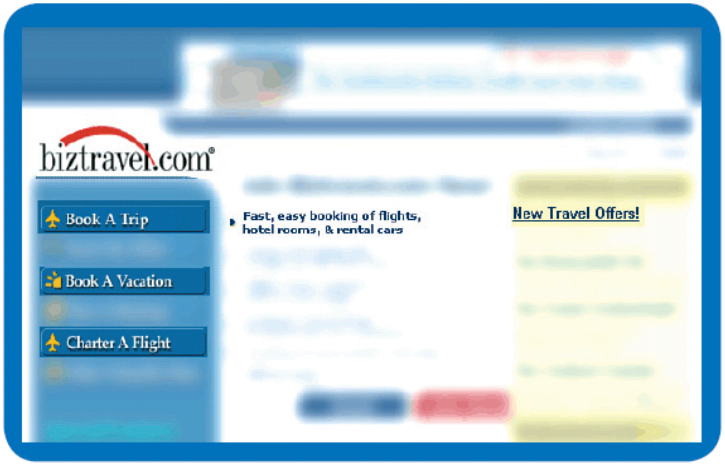
If it doesn't pan out, click the Back button and try again.

Goal Oriented Scanning

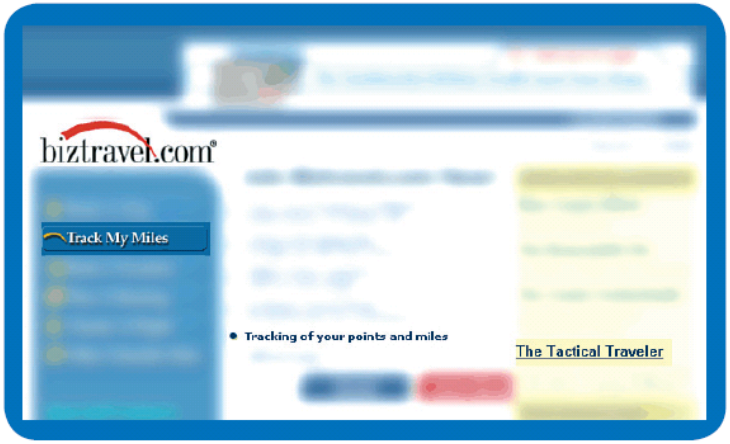




I want to buy a ticket



FACT OF LIFE #1
We don't read pages.
We scan them



How do I check my frequent flyer's miles?



Dibris

Scanning…Is It Real? (heat maps)

- It is
  Tracking eye's focus
  - red areas highest number of hits, then yellow, then blue
- F shaped = reading stops in the middle of sentences



FACT OF LIFE #2
We don't make optimal choices.
We satisfice.

- Satisficing = avg(satisfying, sufficing)
- Why not looking for the best choice?
  - Hurry: "Optimizing is hard, and it takes a long time. Satisficing is more efficient."
  - Not much of a penalty for guessing wrong: a click or two of the Back button
  - Weighing options may not improve our chances on poorly designed interfaces
  - More fun: easier + chance of running into something surprising and good

FACT OF LIFE #3
We don't figure out how things work
We muddle through

- What's a browser?
  - a search tool
  - google



- People don't care to understand as far as it works
- If we find something that works, we stick to it

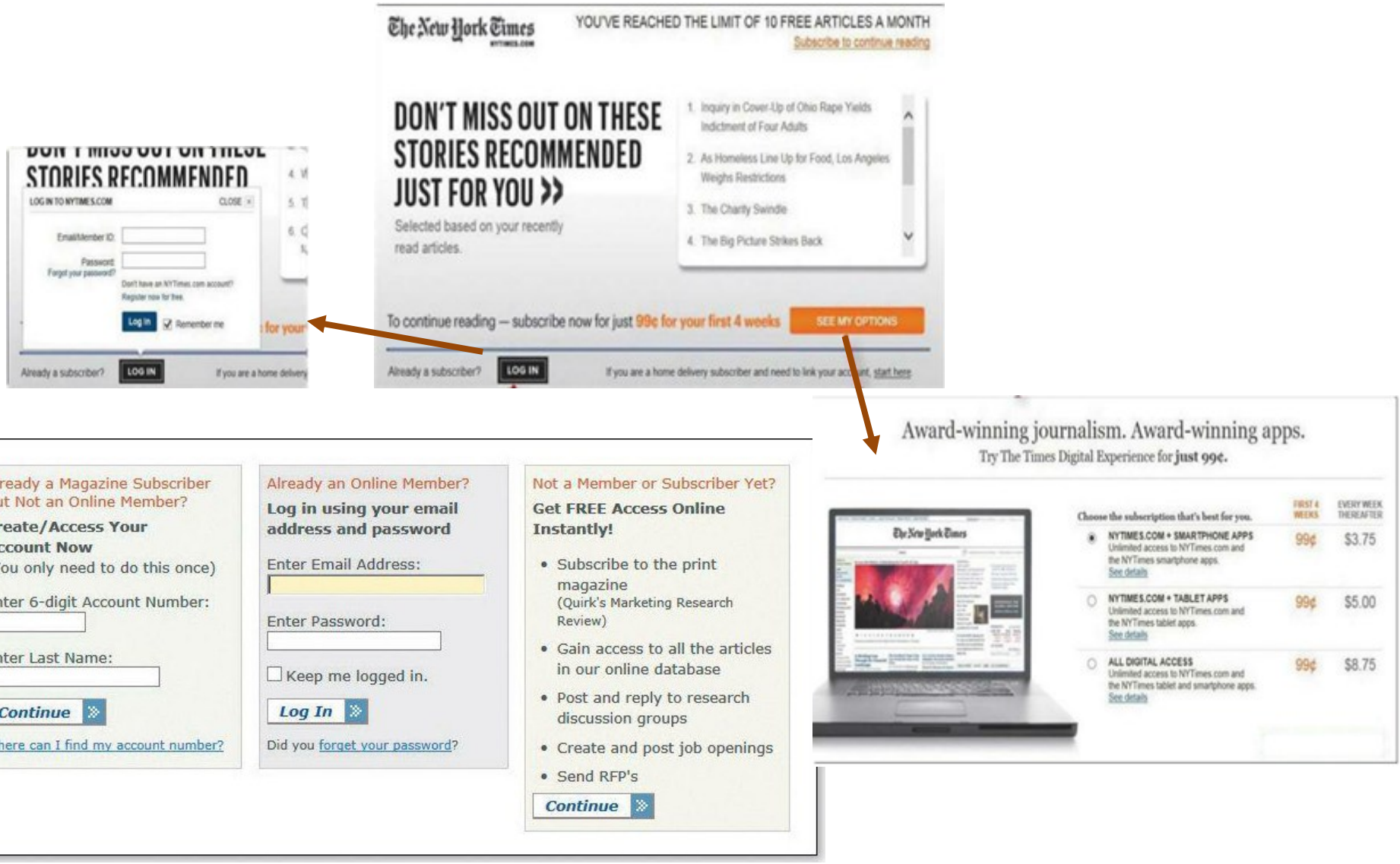
Law#1: Don’t Make Me Think
Designing for Scanning - not Reading

- Take advantage of conventions
- Create effective visual hierarchies
- Break pages up into clearly defined areas
- Make it obvious what’s clickable
- Eliminate distractions
- Format content to support scanning

Law #2: It doesn't matter how many times I have to click, as long as each click is a mindless, unambiguous choice

- Costs of a click
  - load page time
  - effort for understanding/making a decision
  - decision time
- In most cases 3 brainless clicks = 1 motivated click

Making Clicks Painless: an Example

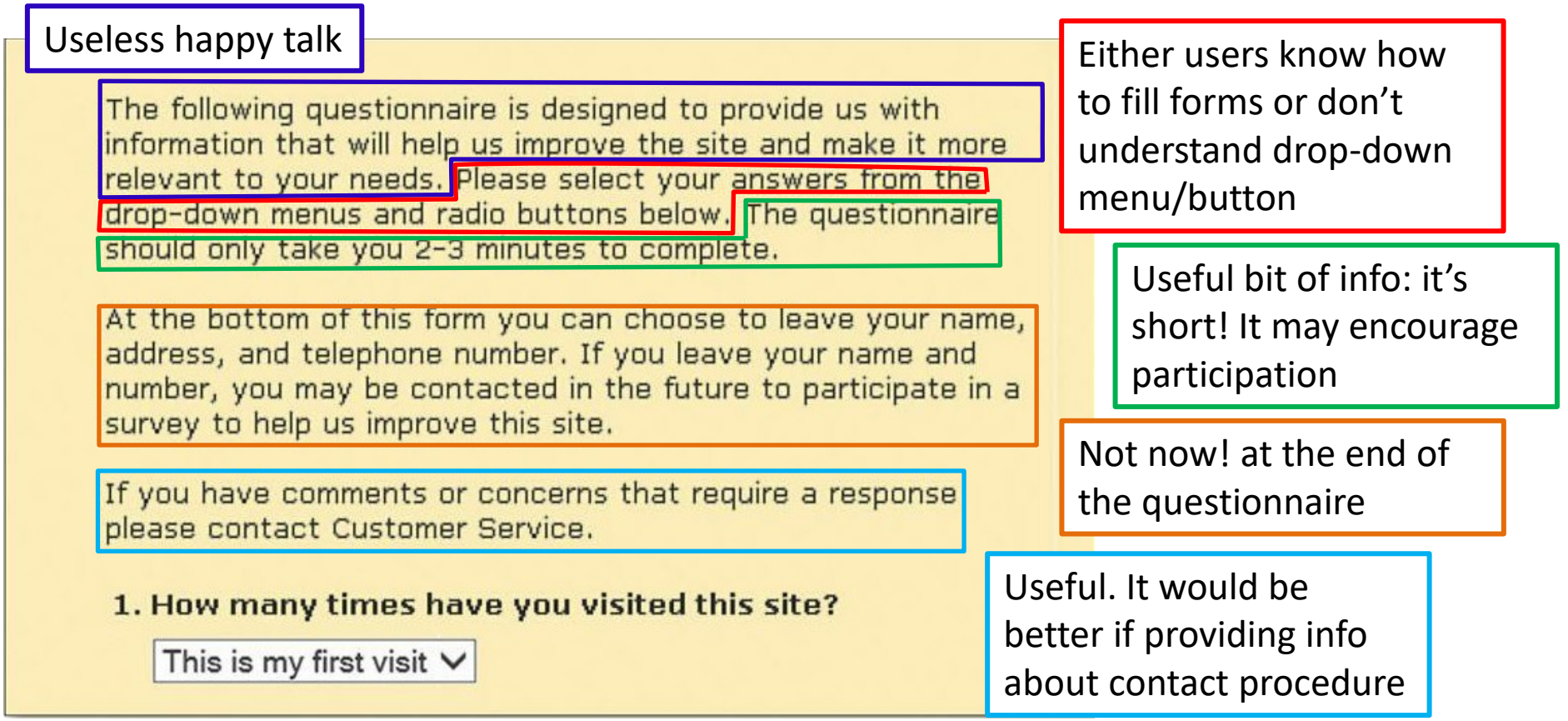


Dibris

Law #3: Get rid of half the words on each page then get rid of half of what's left

- Pros
  - Lower noise level
  - Contents more prominent
  - Pages shorter ⇒ users see more without scrolling
- How
  - Happy talk must die
    users are not here for tea
  - Instructions must die
    everything should be self-explanatory

Merciless Word Pruning: an Example



Please help us improve the site by taking 2-3 minutes to complete this survey.

NOTE: If you have comments or concerns that require a response, don't use this form. Instead, please contact Customer Service.

Dibris

Summary

DON'T MAKE ME THINK

A COMMON SENSE APPROACH TO WEB USABILITY

by STEVE KRUG



"MANY PEOPLE WHO ENCOUNTER PROBLEMS WITH A SITE TEND TO BLAME THEMSELVES AND NOT THE SITE."

THE USABILITY LAWS

1. DON'T MAKE ME THINK!
2. IT DOESN'T MATTER HOW MANY TIMES I HAVE TO CLICK, AS LONG AS EACH CLICK IS A MINDLESS, UNAMBIGUOS CHOICE.
3. GET RID OF HALF THE WORDS ON EACH PAGE, THEN GET RID OF HALF OF WHATS LEFT.

3 FACTS OF WEB LIFE

1. WE DON'T READ PAGES. WE SCAN THEM.
2. WE DON'T MAKE OPTIMAL CHOICES. WE SATISFICE.
3. WE DON'T FIGURE OUT HOW THINGS WORK. WE MUDDLE THROUGH.

How to know if we did things right?

- No way to define automated tests from specifications
  - how to automate sentiments/feelings?
- We need feedback from users $\Rightarrow$ User testing
- Developers are users, wouldn't their opinion suffice?
  - no, if project for specific type of users
  - not the standard users even for people at large
- OK, but developers know how "generic" people are
  - aka "my mom wouldn't like/understand *syndrome*"

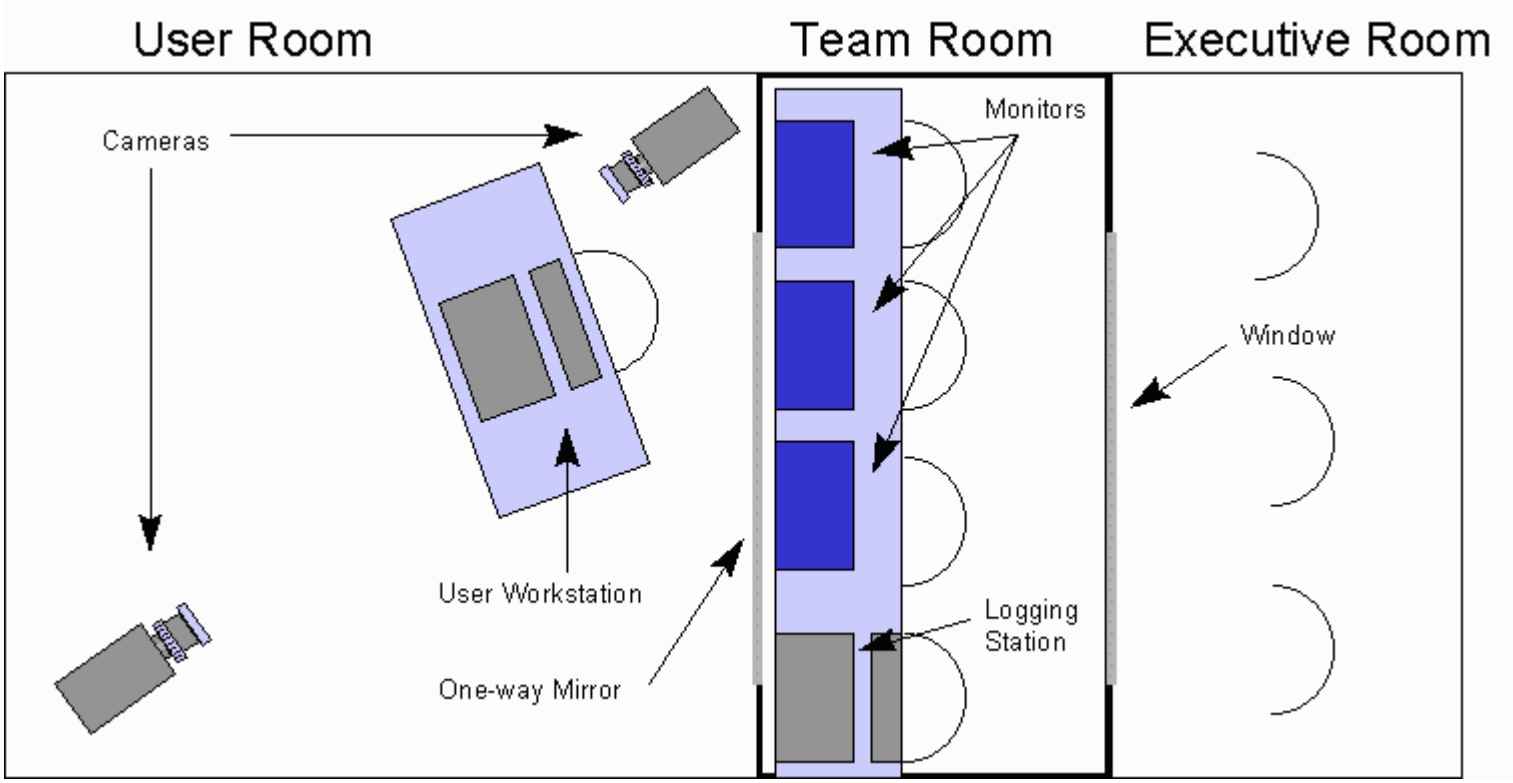
In doubt? Ask the Users!

- Usability Testing
  = watching people trying to use the system while thinking out loud
- Arguments are impartially ruled
  - watching users
  - no personal biases
- Shared experience
- Aha moment: *our users are not like us*
  - see the system through their eyes





Usability Testing



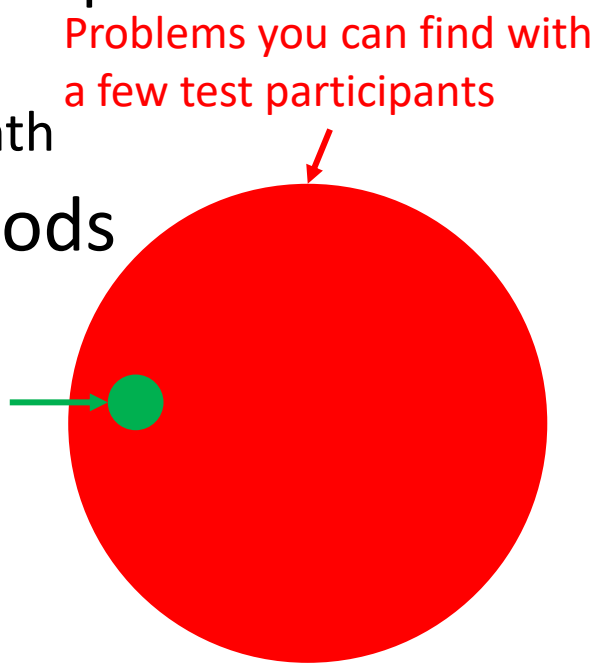
- Lab + experienced professional
  - Same script for several users
    - 8 minimum
  - Record session for further analysis
  - weeks of work
- Analytical report of findings



$5k-$10K

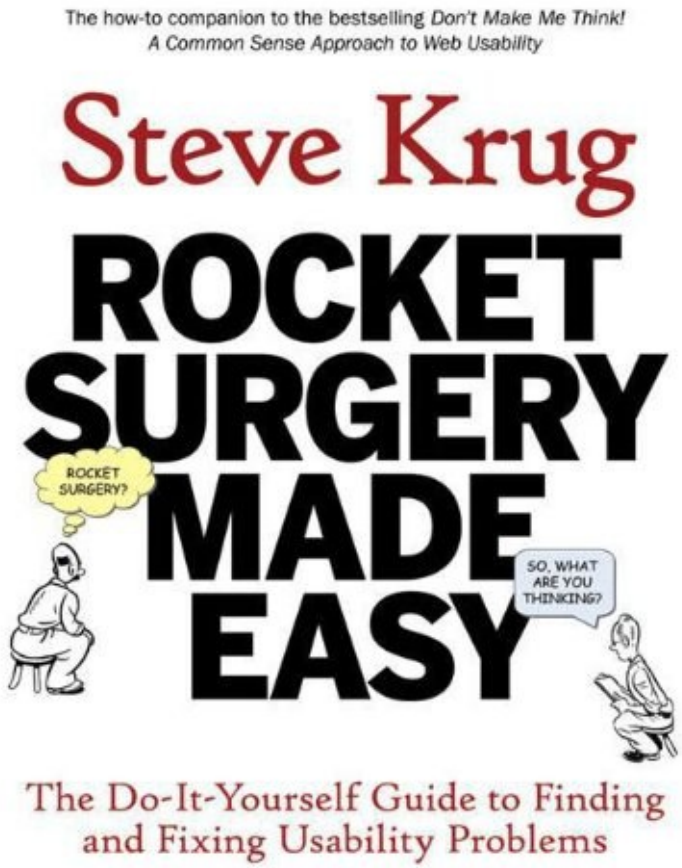
Usability Testing Works

- Uncovers lots of problems quickly ☺
- But...
  - formal testing sessions are expensive
  - feedback delayed by weeks
  - overkill: far less resources needed to find problems than to fix them
    - What's found in a day is not fixed in a month
- Works with *slow* development methods
  - Huge activity
  - Taken once in a while
  - Produces lengthy documentation
    - largely useless



Poor Man Usability Testing

- … or Agile Usability Testing?
- Simplified version
  - no lab/mirror, just camera
    - both record and streaming to other room
  - three people
    - no elaborate recruiting
  - no report
    - debriefing over lunch
- Extremely low cost
- Can be done frequently
  - e.g. half a day each month



How to Manage Users

read users'minds

observe don't lead

When this happens:	Say this:
You're not absolutely sure you know what the user is thinking (see below).	"What are you thinking?" "What are you looking at?" (for variety) "What are you doing now?" (e.g., if you think they're being silent because they're reading)
Something happens that seems to surprise them. For instance, they click on a link and go "Oh" when the new page appears.	"Is that what you expected to happen?"
They're trying to get you to give them a clue. ("Should I use the ___?")	"What would you do if you were at home?" "What would you do if I wasn't here?"
The participant makes a comment, and you're not sure what triggered it.	"Was there something in particular that made you think that?"
The participant suggests concern that he's not giving you what you need.	"No, this is very helpful." "This is exactly what we need."
The participant asks you to explain how something is supposed to work. ("Do these support requests get answered right away?")	"I can't answer that right now, because we need to know what you would do when you don't have somebody around to answer questions for you. But if you still want to know when we're done, I'll be glad to answer it then."
The participant seems to have wandered away from the task.	"What are you trying to do now?"

Dibris

Focus!

- Problem: even the simplified version finds too many problems
- Solution: focus!
  - instruct the observers
  - after each session write down only the three most relevant problems for each user
- Resist temptation to solve easiest first
  - otherwise serious ones will stay there forever
  - exception: so easy that
    - can be fixed by one person in 15'
    - don't need anybody's else approval

Make it a Spectator Sport



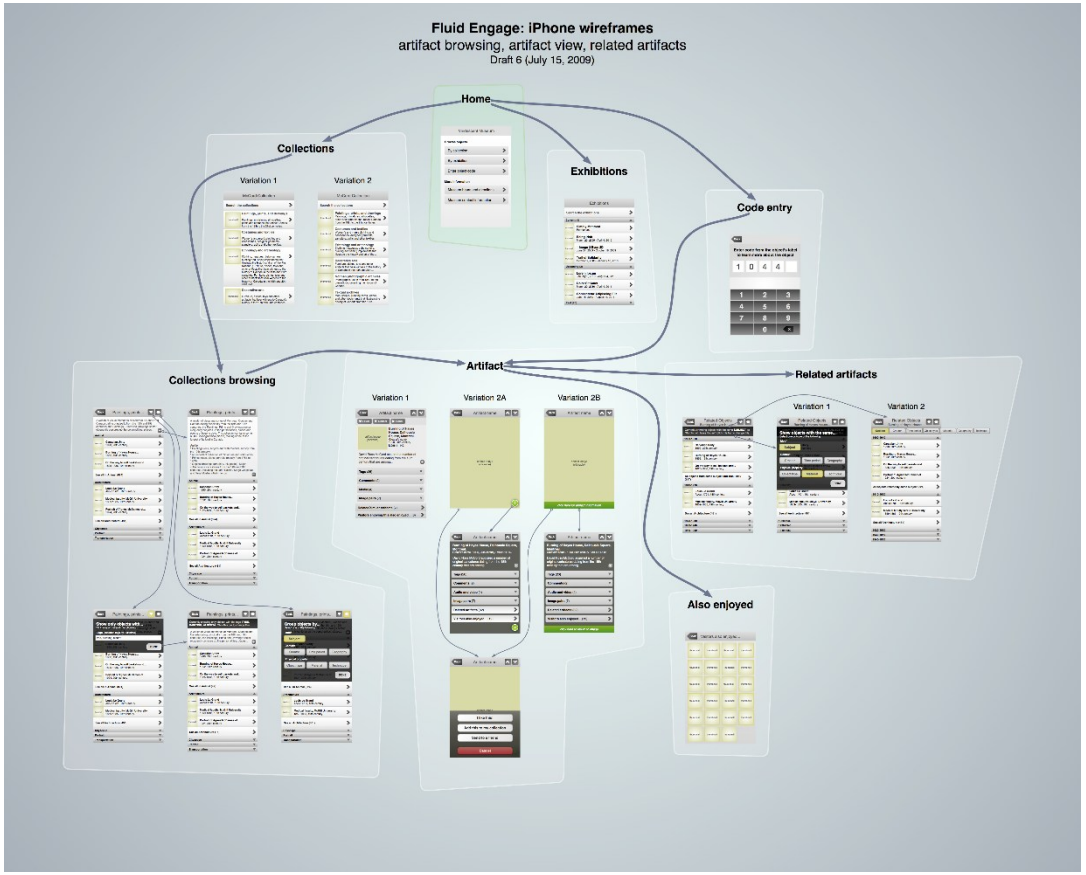
Dibris

Fixing problems

- Always do the least!
- Don’t strive for perfect fix
  - no redesign
  - tweaking
- Tweaking
  - less effort, less time, less cost
  - more likely to happen

When to Test?

- As soon as possible
  - as often as possible
- Test first?
  - preparing task lists
- Paper models
  - less useful if using good developing tool



Dibris